

5 传输层

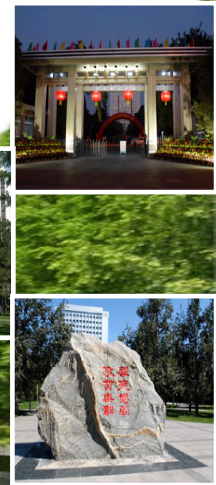
TCP/UDP



北京邮电大学

网络与交换技术全国重点实验室

2026/6/3



主要内容

❖ 传输层的基本原理

❖ TCP/IP 体系中的传输层机制

- ✓ 用户数据报协议 (UDP)
- ✓ 传输控制协议 (TCP)

传输层的基本原理



北京邮电大学

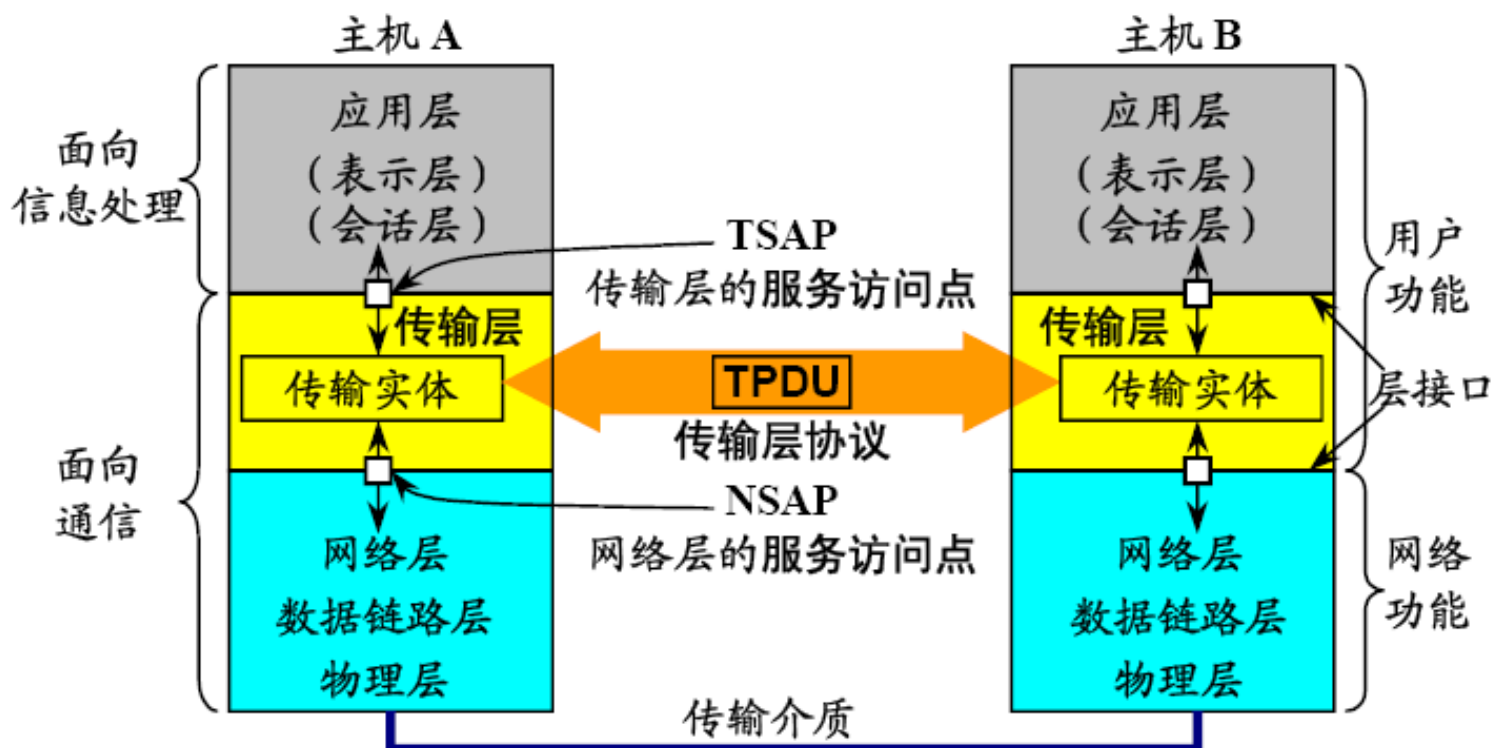
网络与交换技术全国重点实验室

2026/6/3



传输层

- ❖ 从通信和信息处理的角度看，传输层向它上面的应用层提供通信服务，它属于面向通信部分的最高层，同时也是用户功能中的最低层。



传输层和网络层的主要区别

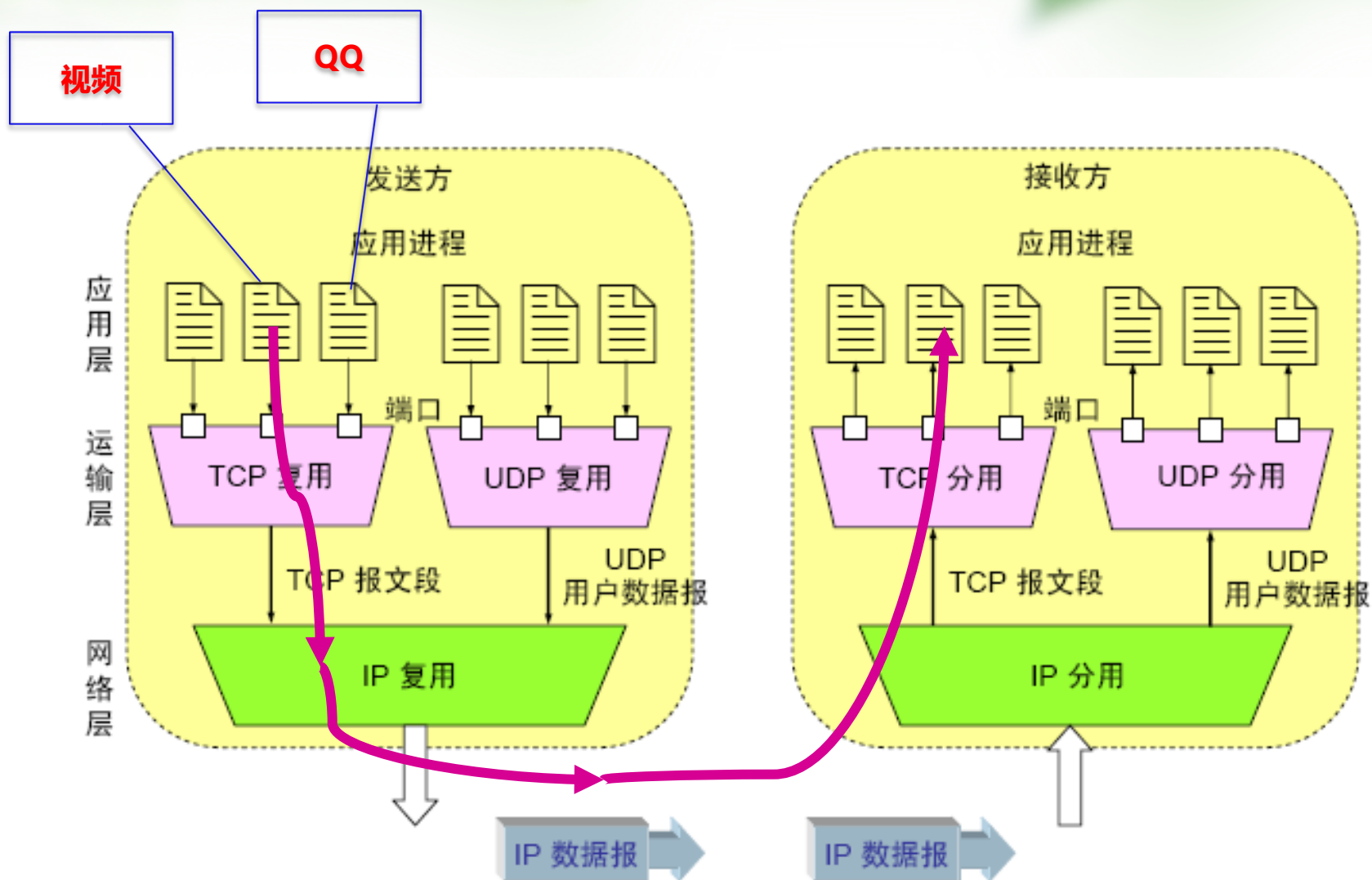
❖ 传输层和网络层的主要区别

- ✓ 传输层为应用进程之间提供逻辑通信
- ✓ 网络层为主机之间提供逻辑通信

❖ 传输层的主要功能

- ✓ 传输层为应用进程之间提供端到端的逻辑通信
 - ✉ 传输层的一个很重要的功能就是复用和分用。
 - ✉ “逻辑通信”的意思是：传输层之间的通信好像是沿水平方向传送数据。但事实上这两个传输层之间并没有一条水平方向的物理连接。
- ✓ 传输层还要对收到的报文进行差错检测。

网络应用：进程间通信



进程之间的通信

❖ 引入传输层的原因

- ✓ 消除网络层的不可靠性，提供从源端主机到目的端主机可靠的信息传输；
 - ✉ 用户无法对子网加以控制，故需在网络层之上设计一层来改善服务质量。
 - ✉ 传输层要对收到的报文进行差错检测。（在网络层，IP数据报只校验首部的差错，而不检查数据部分）
- ✓ 提供从源端主机到目的端主机与实际使用网络无关的信息传输。
 - ✉ 传输层向高层用户屏蔽了下面通信子网的细节（网络拓扑及采用的协议等）

传输层和网络层的主要区别

❖ 网络层

- ✓ 对高层用户屏蔽了底层网络在连接、可达性、拓扑、传输技术上的差异

❖ 传输层

- ✓ 对高层用户屏蔽了通信子网在传输质量上（服务质量QoS）上的差异
- ✓ 向上层提供统一的服务质量的数据传输服务：差错检测/处理

传输层的基本功能

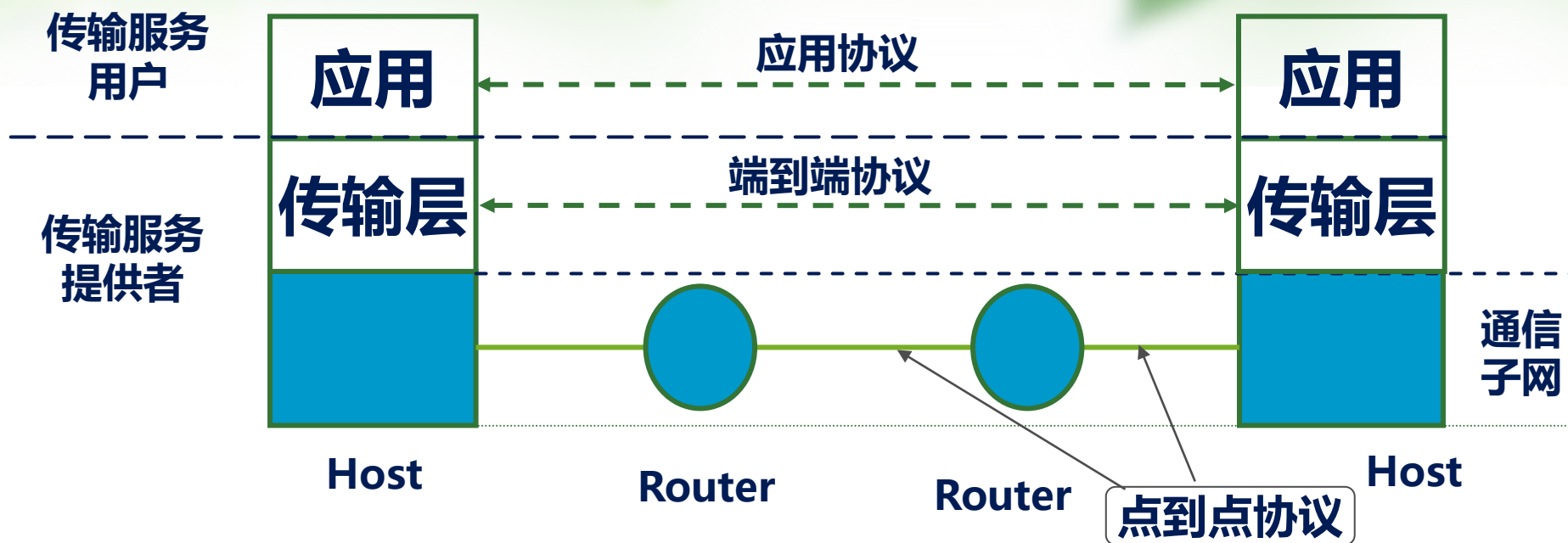
❖ TCP/IP 体系结构中传输层的基本功能

- ✓ 为信源结点和目的结点间的通信提供端到端的数据传输
- ✓ 而通信子网只能提供相邻结点之间的点到点传输

❖ 服务 (Service) 与服务质量 (QoS)

- ✓ 相邻网络层次间的界面反映了相邻层次间的关系
- ✓ 服务：网络中某层向其相邻高层提供的一组操作或接口
- ✓ 服务具有单向性（低层为服务的提供者，高层为服务的用户）
- ✓ 服务的表现形式：原语 (primitive) ， 如系统调用
- ✓ 各层次提供的服务具有不同的服务质量：
 - ☒ 是否面向连接、连接建立和释放的时间、连接建立失败的概率
 - ☒ 传输时延及其抖动、吞吐率、误码率
- ✓ 高层提供服务的 QoS 总是比低层服务的 QoS 更完善

通信子网服务与传输服务



❖ 传输层服务

- ✓ 屏蔽通信子网QoS细节，增强通信子网服务的 QoS
- ✓ 传输服务提供者：网络分层模型中传输层以下的部分
- ✓ 传输服务用户：传输层以上的应用

传输层服务

❖ 传输服务需要解决的问题

✓ 提供的 QoS

✉ 提供面向连接的传输服务，还是无连接的传输服务？

✓ 传输层地址设计 —— 服务访问点（TSAP）的地址标识

✓ 连接的管理

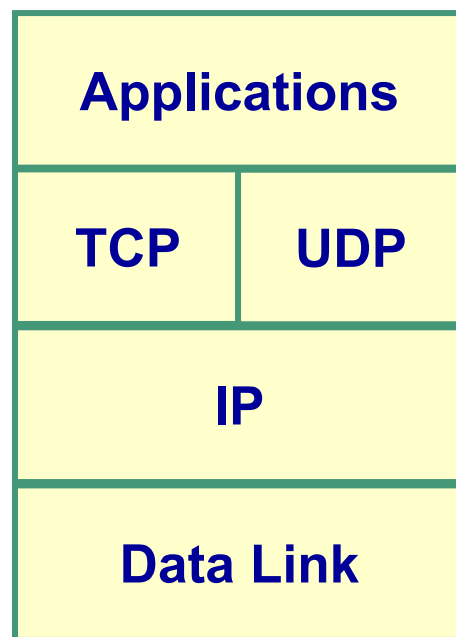
✓ 差错控制与流量控制

✓

❖ TCP/IP 体系中的传输层

✓ **UDP** – User Datagram Protocol

✓ **TCP** – Transport Control Protocol



UDP与TCP

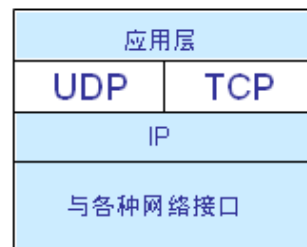
❖ UDP(用户数据报协议User Datagram Protocol)

- ✓ UDP 在传送数据之前不需要先建立连接。
- ✓ 对方的传输层在收到 UDP 报文后，不需要给出任何确认。
- ✓ 虽然 UDP 不提供可靠性，但效率更高

❖ TCP(传输控制协议Transmission Control Protocol)

- ✓ 保证可靠性
- ✓ TCP 则提供面向连接的服务
- ✓ TCP 不提供广播或多播服务
- ✓ TCP增加了许多的开销

✉ 使协议数据单元的首部增大很多，处理效率相对较低



传输层的地址——端口

❖ UDP/TCP 中采用端口 (port) 来标识 TSAP

- ✓ 传输端口代表 TCP/UDP 的传输服务访问点 TSAP
- ✓ 在进程通信中标识相互通信的进程
 - ✉ 通信的对端进程地址可表示为: (IP address , port)

❖ 传输端口的绑定 (binding)

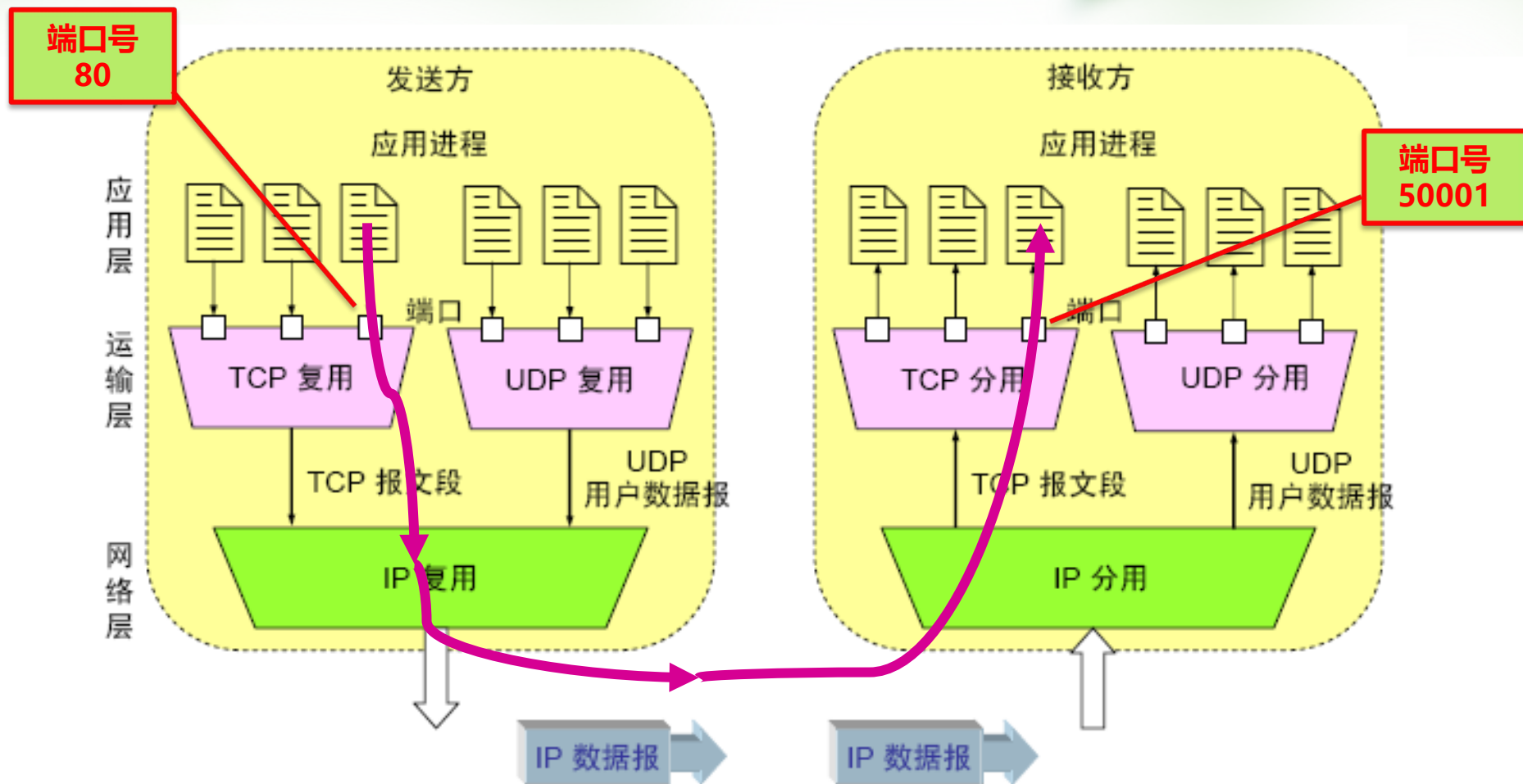
- ✓ 进程在某个传输端口进行数据传输前, 必须首先通过系统调用与该端口建立绑定关系

❖ UDP/TCP 的传输端口号 (port number)

- ✓ 端口号用于标识 UDP/TCP 的传输端口
- ✓ UDP/TCP 协议各分别可以提供最多 64K 个传输端口



端口在进程间通信中所起的作用



端口

❖ 为什么需要端口？

- ✓ 让应用层的各种应用进程都能将其数据通过端口向下交付给传输层，以及让传输层知道应当将其报文段中的数据向上通过端口交付给应用层相应的进程。

- ✓ 端口就是传输层服务访问点TSAP

❖ 端口用一个 16 位端口号进行标志。

❖ 端口号只具有本地意义

- ✓ 端口号只是为了标志本计算机应用层中的各进程。在因特网中不同计算机的相同端口号是没有联系的。

传输端口的分配

❖ 进程通信时，必须了解对端进程的地址 (IP + port)

✓ 主要问题：如何了解对端进程所使用的端口号？

❖ 端口分配方式

✓ 全局统一分配端口号

✓ 动态绑定方式（本地分配）

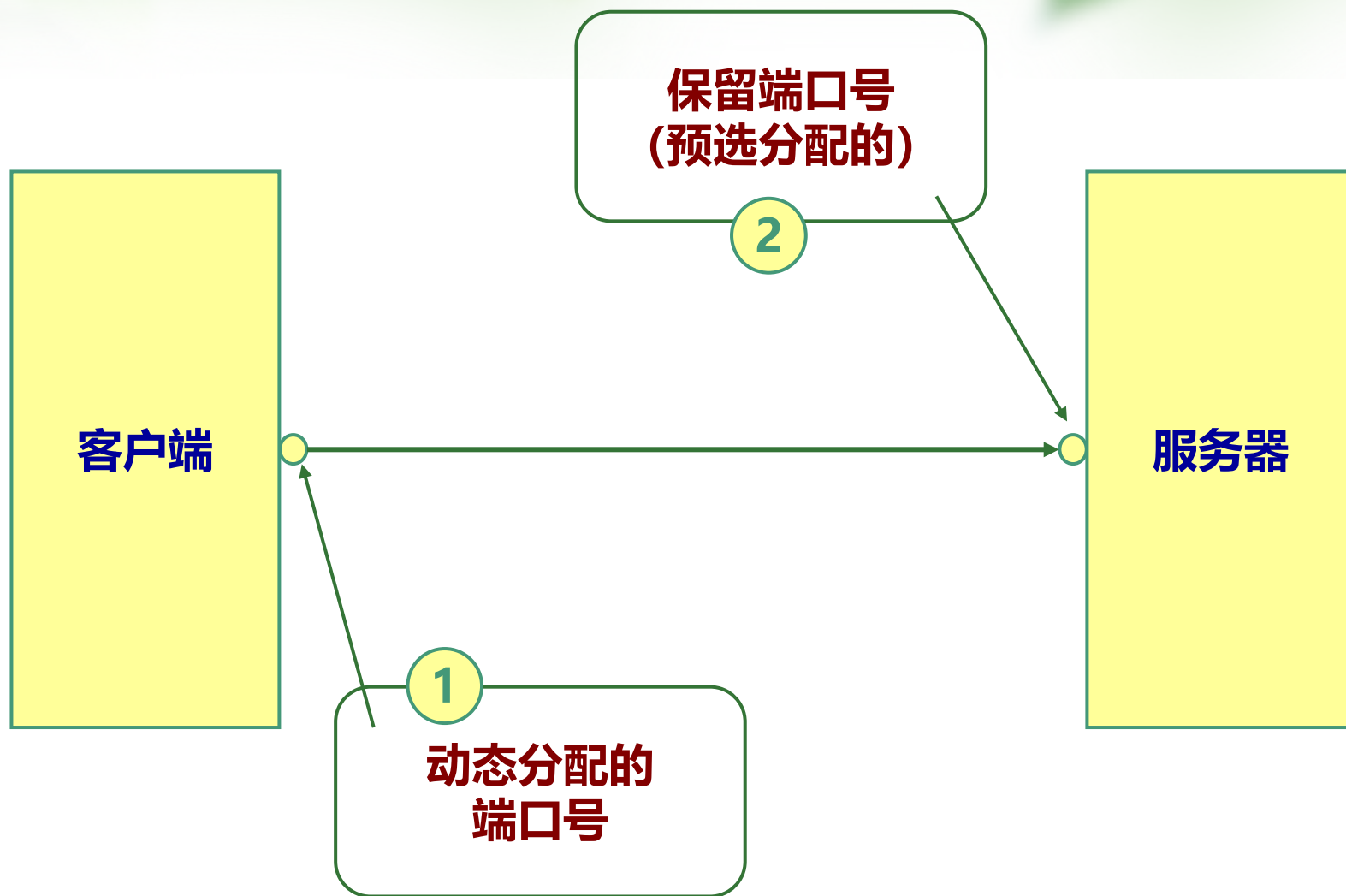


传输端口的分配

❖ TCP/IP 系统中端口分配方法

- ✓ 应用进程通信采用“客户-服务器” (client-server) 模式
- ✓ 将传输端口划分为两类：保留端口和自由端口
 - ✉ 保留端口(well-known port)：为服务进程全局分配的端口
 - ✉ 自由端口是在进程需要进行通信时，由本地进行动态分配的
 - ✉ 客户进程首先动态申请一个本地自由端口号，再通过服务进程所公布的保留端口与服务器进程建立联系

传输端口的分配



端口号的分类

❖ 服务器端使用的端口号

✓ 熟知端口（保留端口）

- ☒ 数值一般为 0~1023。
- ☒ 由互联网名称与数字地址分配机构（ICANN）确定，如HTTP用80、FTP用21、DNS用53、SMTP用25、TELNET用23、.....

✓ 登记端口号

- ☒ 数值为1024~49151，为没有熟知端口号的应用程序使用的。

❖ 客户端口号或短暂端口号

- ✓ 数值为49152~65535，留给客户进程选择暂时使用。当服务器进程收到客户进程的报文时，就知道了客户进程所使用的动态端口号。通信结束后，这个端口号可供其他客户进程以后使用。

❖ 登记端口号与客户端口号也可以统称为一般端口号。

表 5-1 使用 UDP 和 TCP 协议的各种应用和应用层协议

应用	应用层协议	运输层协议
域名转换	DNS (域名系统)	UDP
文件传送	TFTP (简单文件传送协议)	UDP
路由选择协议	RIP (路由信息协议)	UDP
IP 地址配置	DHCP (动态主机配置协议)	UDP
IP 电话	专用协议	UDP
流式多媒体通信	专用协议	UDP
多播	IGMP (网际组管理协议)	UDP
电子邮件	SMTP (简单邮件传送协议)	TCP
万维网	HTTP (超文本传送协议)	TCP
文件传送	FTP (文件传送协议)	TCP

常用的保留端口号

UDP 保留 端口号

7	ECHO	回送
37	TIME	时间
42	NAMESERVER	主机名字服务器
53	DOMAIN	域名服务器
67	BOOTPS	启动协议服务
69	TFTP	简单文件传输
161	SNMP	SNMP 网络监控
.....		

TCP 保留 端口号

20	FTP-DATA	文件传输协议 (数据连接)
21	FTP	文件传输协议 (控制连接)
23	TELNET	远程登录终端
37	TIME	时间
43	NICNAME	whois 程序
79	FINGER	finger 程序
80	HTTP	WEB 服务
.....		



用户数据报协议 (UDP)

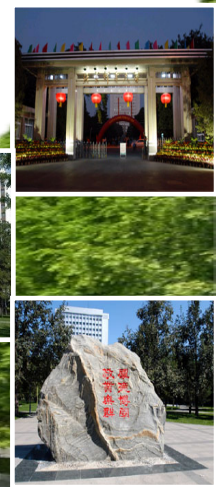
User Datagram Protocol



北京邮电大学

网络与交换技术全国重点实验室

2026/6/3



UDP 协议

- ❖ 以实现效率为首要目标，具有良好的实时性
- ❖ 提供无连接、不可靠的传输服务
- ❖ 会出现分组丢失、重复、乱序
- ❖ 应用程序需要负责传输可靠性方面的所有工作



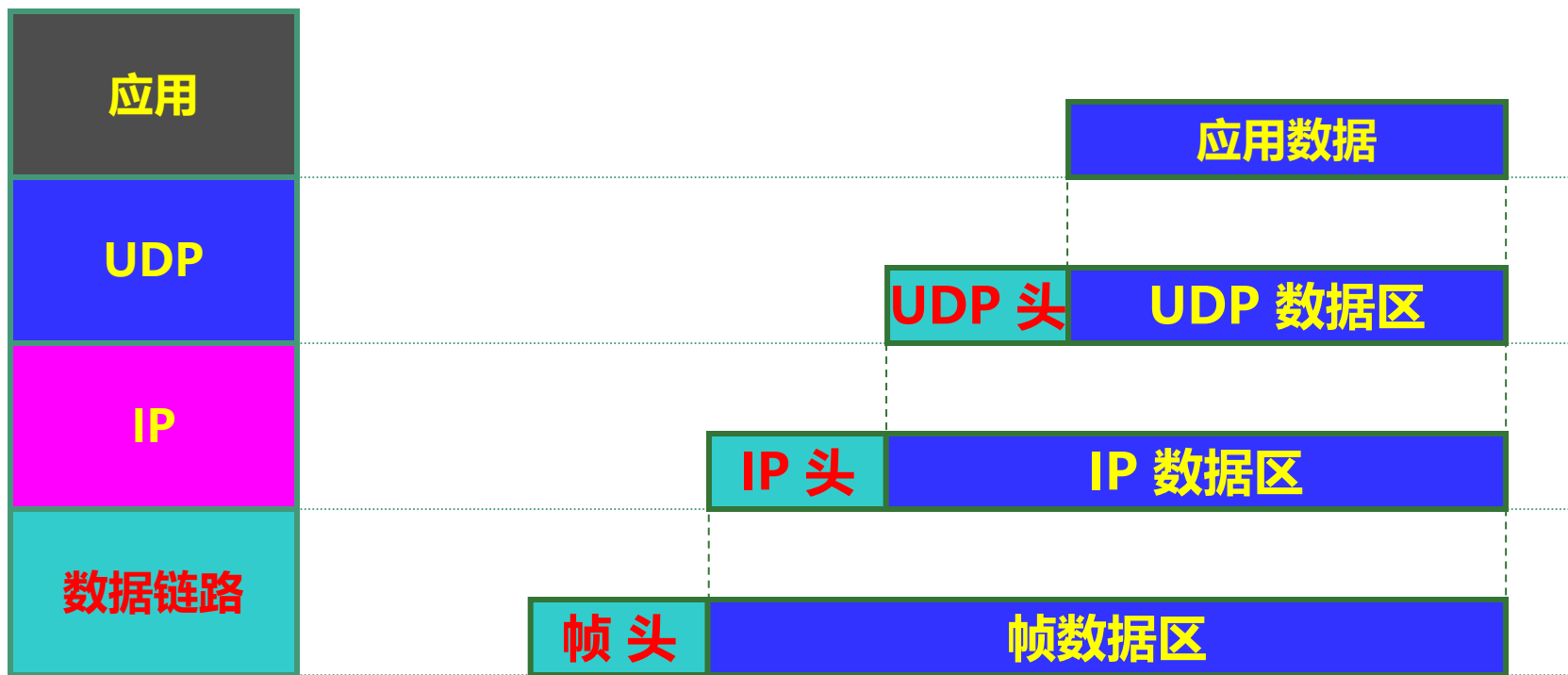
UDP 概述

- ❖ UDP 只在 IP 的数据报服务之上增加了很少一点的功能，即端口的功能和差错检测的功能。
- ❖ UDP 的主要特点
 - ✓ UDP 是无连接的，即发送数据之前不需要建立连接。
 - ✓ UDP 使用尽最大努力交付，即不保证可靠交付，同时也不使用拥塞控制。
 - ✓ UDP 是面向报文的。（对应用层交下来的报文，既不合并，也不拆分）
 - ✓ UDP 没有拥塞控制，很适合多媒体通信的要求。
 - ✓ UDP 支持一对一、一对多、多对一和多对多的交互通信。
 - ✓ UDP 的首部开销小，只有 8 个字节。

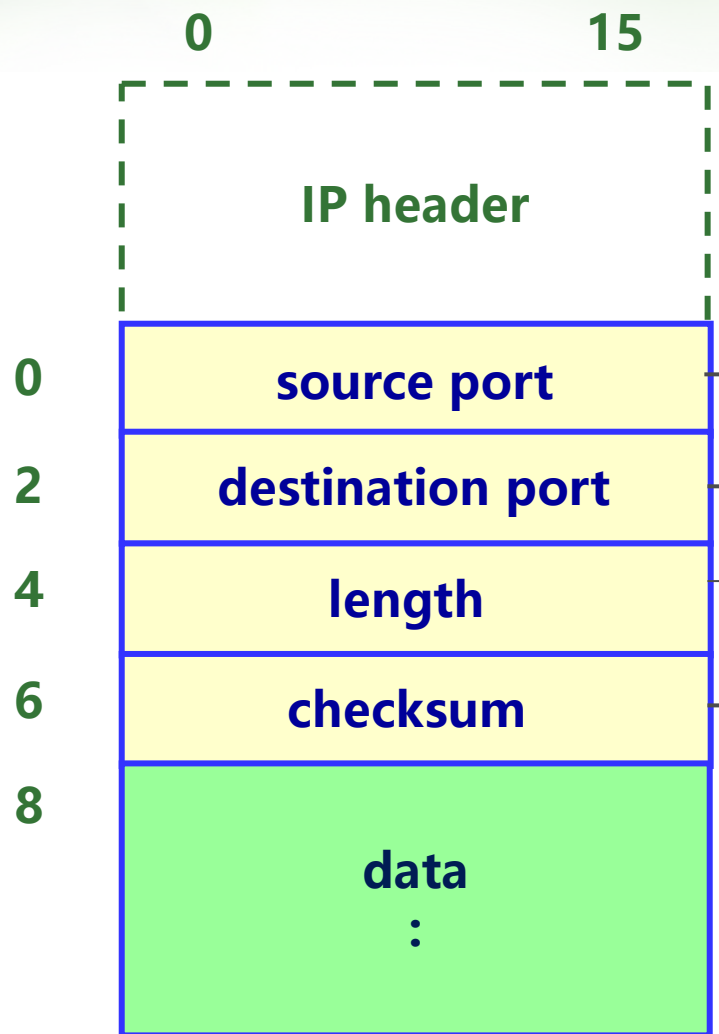
UDP 协议封装

- ❖ UDP 数据报由两部分构成：UDP 报头和数据区
- ❖ UDP 报文是封装在 IP 分组中进行传送的

概念分层



UDP 数据报的格式



UDP 源端口:

发送端的 UDP 端口号; 当不需要对端返回数据时, 该字段为 0。

UDP 目的端口:

接收端的 UDP 端口号。

数据报长度:

以字节计算的整个数据报的长度, 最小值 8 字节 (只含 UDP 头)。

数据报校验和 (可选):

0: 表示未选用校验功能;
其它值表示数据报的校验和, 若该字段为全 1 则表示校验和为 0。

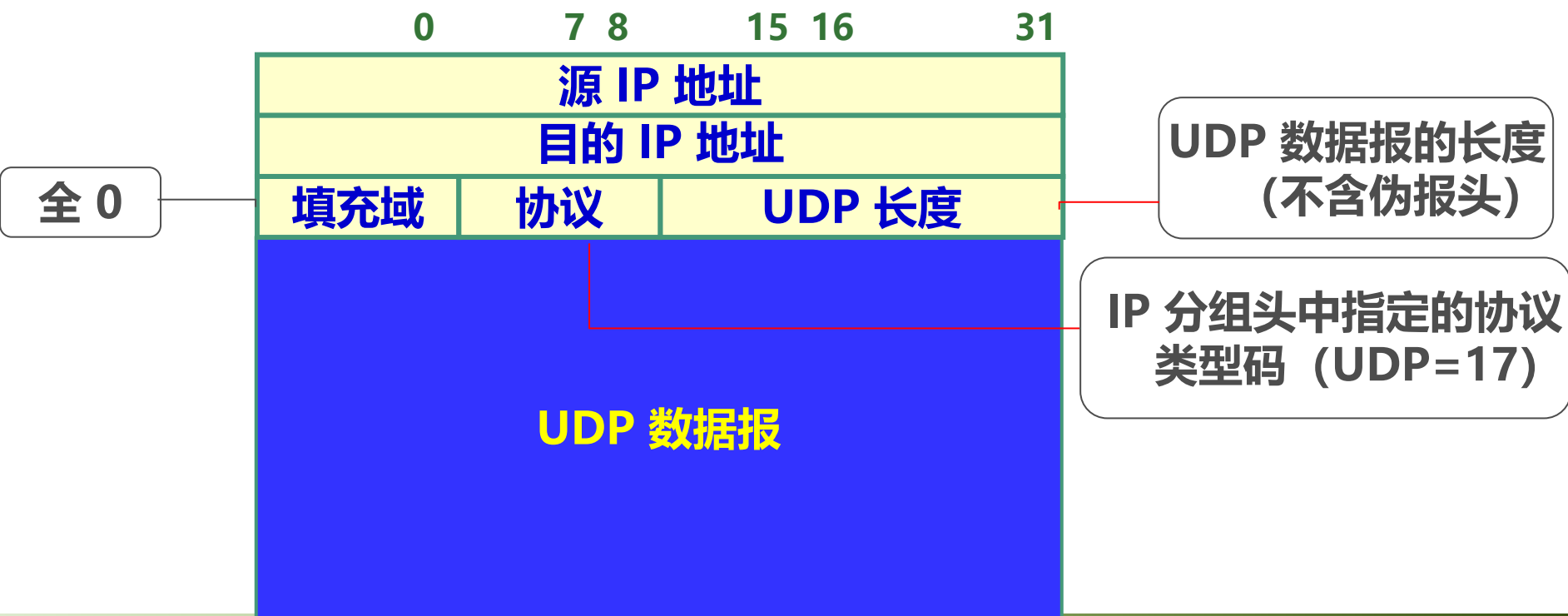
UDP 数据报的校验

❖ UDP 数据报校验是一项可选的功能

✓ 用户禁止该功能可以进一步提高通信的效率

❖ UDP 校验和的计算方法：与 IP 分组头的校验相同

✓ 校验和计算：除覆盖数据报外，还覆盖一个 UDP 伪报头



UDP 数据报的校验

❖ UDP 数据报校验是一项可选的功能

✓ 用户禁止该功能可以进一步提高通信的效率

❖ UDP 校验和的计算方法：与 IP 分组头的校验相同

✓ 校验和计算：除覆盖数据报外，还覆盖一个 UDP 伪报头

➤ 说明

- ❑ 伪报头并非 UDP 数据报中实际的有效成分

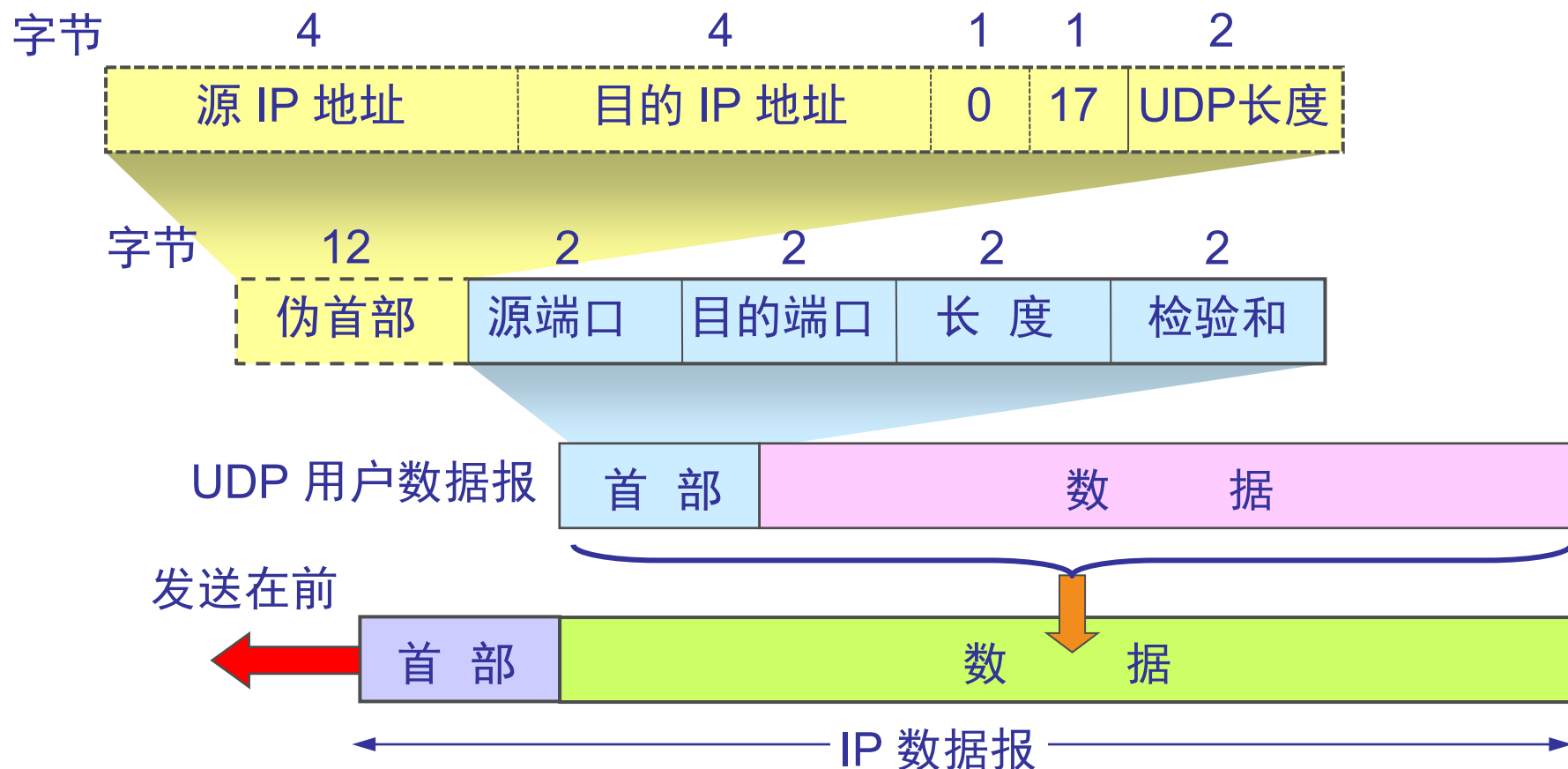
- ❑ 伪报头是一个虚拟的数据结构：

其中的信息是从数据报所在 IP 分组头的分组头中提取的

- ❑ 使用伪报头是为了验证 UDP 数据报是否正确地传到了目的系统中

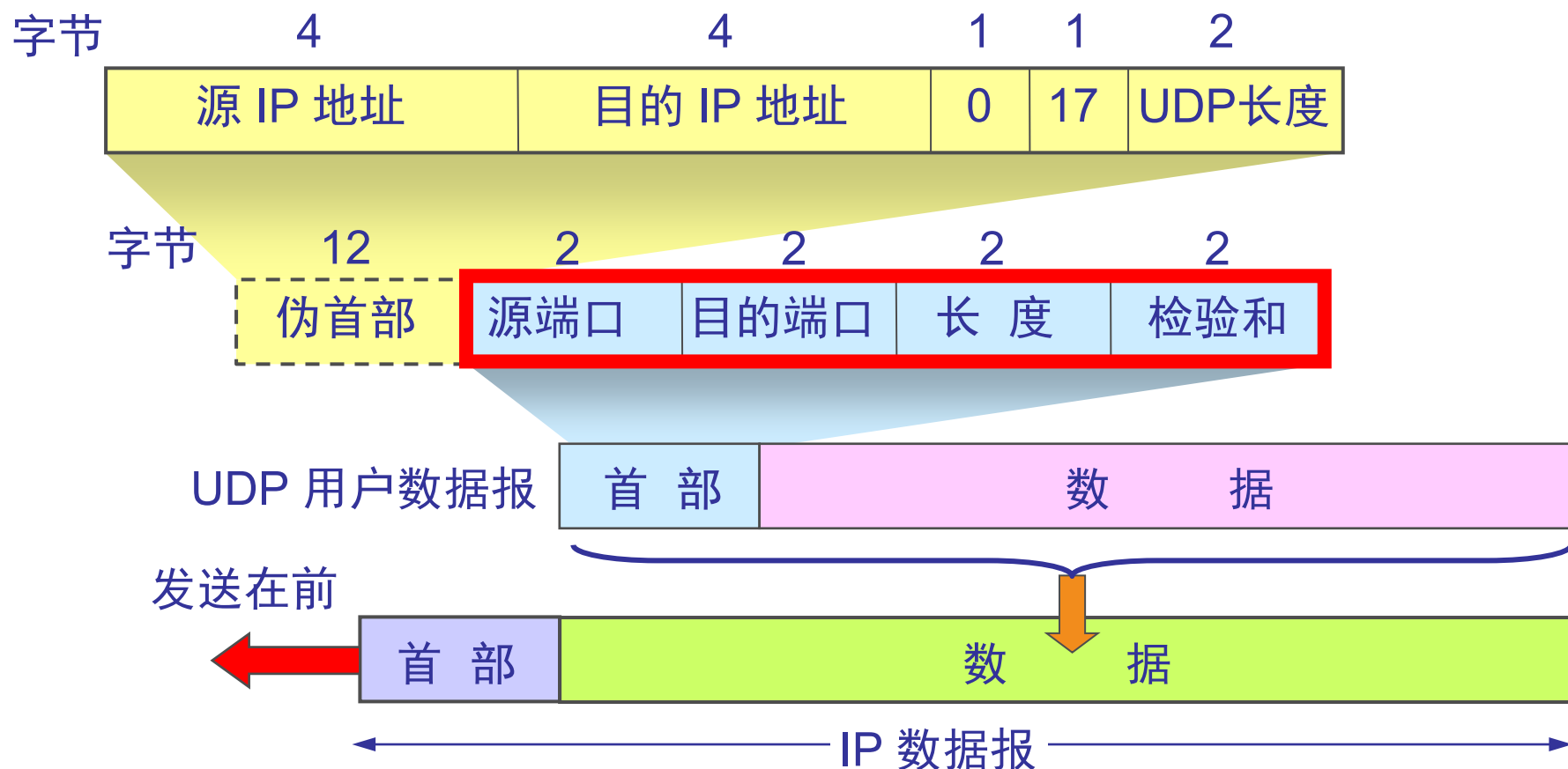
- ❑ 伪报头的采用在一定程度上违反了网络结构分层的原则

UDP 的报文格式 (小结)

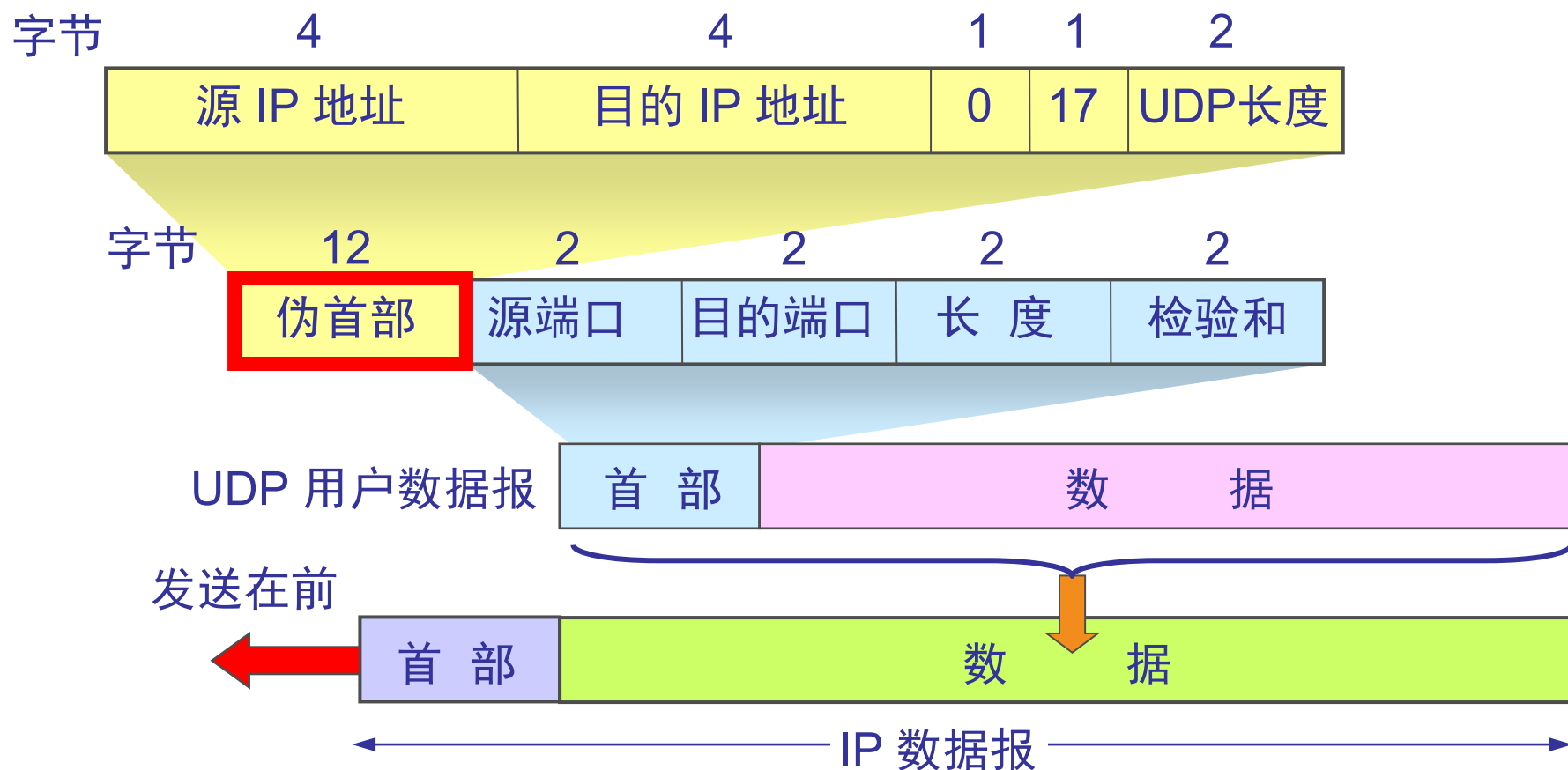


UDP用户数据报：首部、数据区

首部 8 个字节，由 4 个字段组成，每个字段都是两个字节。



在计算检验和时，临时把“伪首部”和 UDP 用户数据报连接在一起。伪首部仅仅是为了计算检验和。



UDP 基本工作过程

❖ UDP 数据报的发送和接收通过 UDP 端口 实现

- ✓ 端口是一个可读写的结构，具有内部的报文缓冲区

❖ 数据报发送

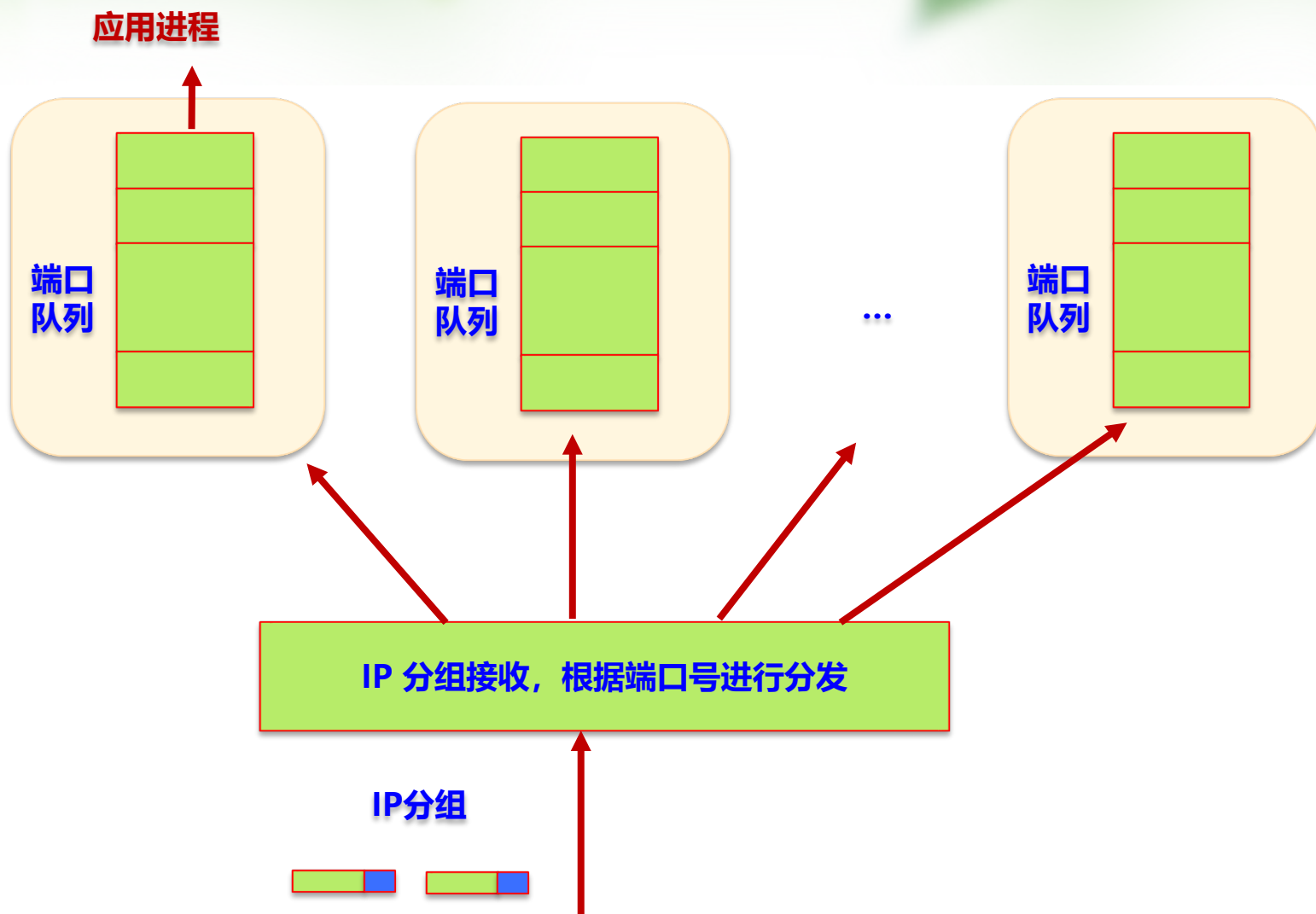
- ✓ UDP 软件将用户数据封装在 UDP 数据报中
- ✓ 转交给 IP 软件，进行 IP 封装和转发

❖ 数据报的接收

- ✓ IP 层接收到 UDP 数据报，提交给 UDP 软件的各端口
- ✓ 端口判断该报文的目的端口号是否与当前端口匹配
- ✓ 若匹配成功，将该数据报保存到相应端口的接收队列中；（若队列已满，则丢弃该数据报）
- ✓ 若未匹配，则丢弃该数据报，同时向源端发送 “端口不可达” 的 ICMP 包



UDP 工作流程



传输控制协议

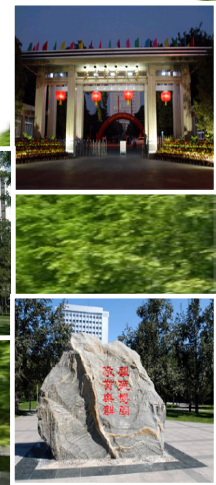
TCP – Transport Control Protocol



北京邮电大学

网络与交换技术全国重点实验室

2026/6/3



TCP 的可靠传输服务特性

❖ TCP 向应用程序提供可靠的传输服务

- ✓ 着重解决传输的可靠性问题 (分组丢失、失序
- ✓ 适用于计算机之间的大量数据传输
- ✓ 协议复杂、效率较低 (与 UDP 相比)

❖ TCP 可靠传输服务接口的特征:

- ✓ 面向数据流
- ✓ 有缓存的传送
- ✓ 全双工连接
- 虚电路连接
- 无结构的数据流

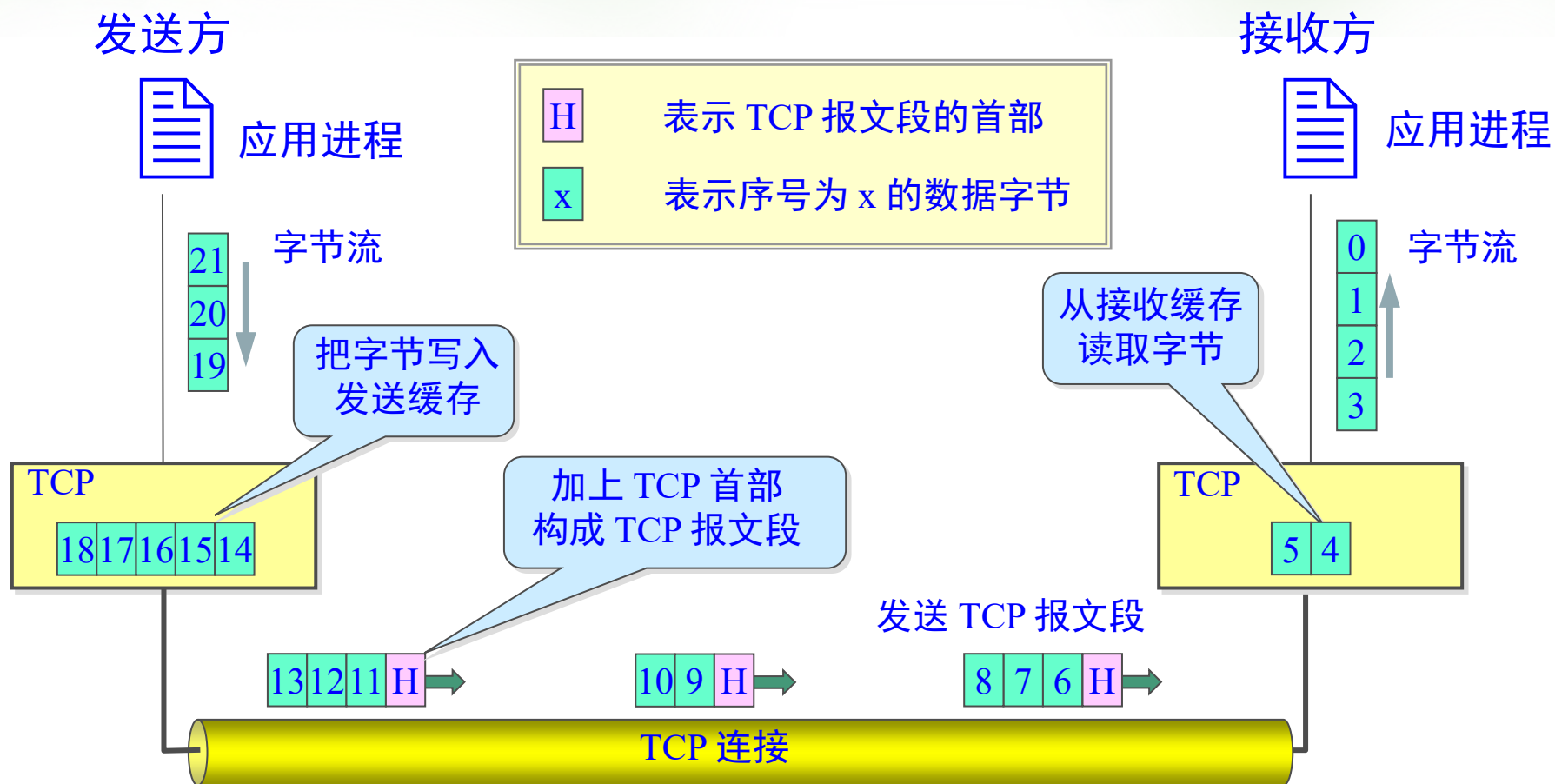
❖ TCP 的可靠性机制

- ✓ 数据确认和重传
- ✓ 滑动窗口进行流量控制、防止缓冲溢出

TCP 最主要的特点

- ❖ TCP 是面向连接的传输层协议。
- ❖ 每一条 TCP 连接只能有两个端点(endpoint)，每一条 TCP 连接只能是点对点的（一对一）。
- ❖ TCP 提供可靠交付的服务。
- ❖ TCP 提供全双工通信。
- ❖ 面向字节流。

TCP 面向流的概念



TCP特点

- ❖ TCP 连接是一条虚连接而不是一条真正的物理连接。
- ❖ TCP 对应用进程一次把多长的报文发送到TCP 的缓存中是不关心的。
- ❖ TCP 根据对方给出的窗口值和当前网络拥塞的程度来决定一个报文段应包含多少个字节（UDP 发送的报文长度是应用进程给出的）。
- ❖ TCP 可把太长的数据块划分短一些再传送。TCP 也可等待积累有足够多的字节后再构成报文段发送出去。



TCP 传输端口与连接

❖ TCP 采用传输端口来标识 TCP 连接

- ✓ TCP 协议提供面向连接的虚电路服务
- ✓ 传输端口（即传输层TCP地址，TSAP）标识了通信进程
- ✓ 每个进程可以多连接通信
- ✓ 端口号被用于标识同一个系统中的多个通信对端进程
- ✓ TCP 可提供基于传输端口的数据复用
- ✓ 由于进程通信是通过 TCP 连接实现的，连接的两个端点（也就进程）可用整数对 (host IP , port) 来标识
- ✓ 给定连接的两个端点，就可以唯一地标识一个 TCP 连接

TCP 的连接

❖ TCP 把连接作为最基本的抽象，每一条 TCP 连接有两个端点。

✓ TCP 连接的端点称为套接字(socket)或插口

✓ IP+端口号拼接构成了套接字。

✉ 套接字 socket = (IP地址: 端口号)

❖ 每一条 TCP 连接唯一地被通信两端的两个端点（即两个套接字）所确定。即：

✓ TCP 连接 ::= {socket1, socket2} = {(IP1: port1), (IP2: port2)}

TCP 传输端口与连接

❖ 在 TCP 中，用户收发数据是通过连接来进行的

- ✓ 与 UDP 不同（其报文收发仅通过协议端口）
- ✓ 由于 TCP 使用两个端点来标识连接，故一个主机上的某个 TCP 端口号可被多个连接所共享



TCP 的传输连接管理

传输连接的三个阶段

- ❖ 传输连接就有三个阶段，即：**连接建立、数据传送和连接释放**。传输连接的管理就是使传输连接的建立和释放都能正常地进行。
- ❖ 连接建立过程中要解决以下三个问题：
 - ✓ 要使每一方能够确知对方的存在。
 - ✓ 要允许双方协商一些参数（如最大报文段长度，最大窗口大小，服务质量等）。
 - ✓ 能够对传输实体资源（如缓存大小，连接表中的项目等）进行分配。
- ❖ TCP 连接的建立都是采用**客户-服务器方式**。
 - ✓ 主动发起连接建立的应用进程叫做**客户(client)**。
 - ✓ 被动等待连接建立的应用进程叫做**服务器(server)**。



TCP 报文段的格式

❖ TCP 报文段的结构

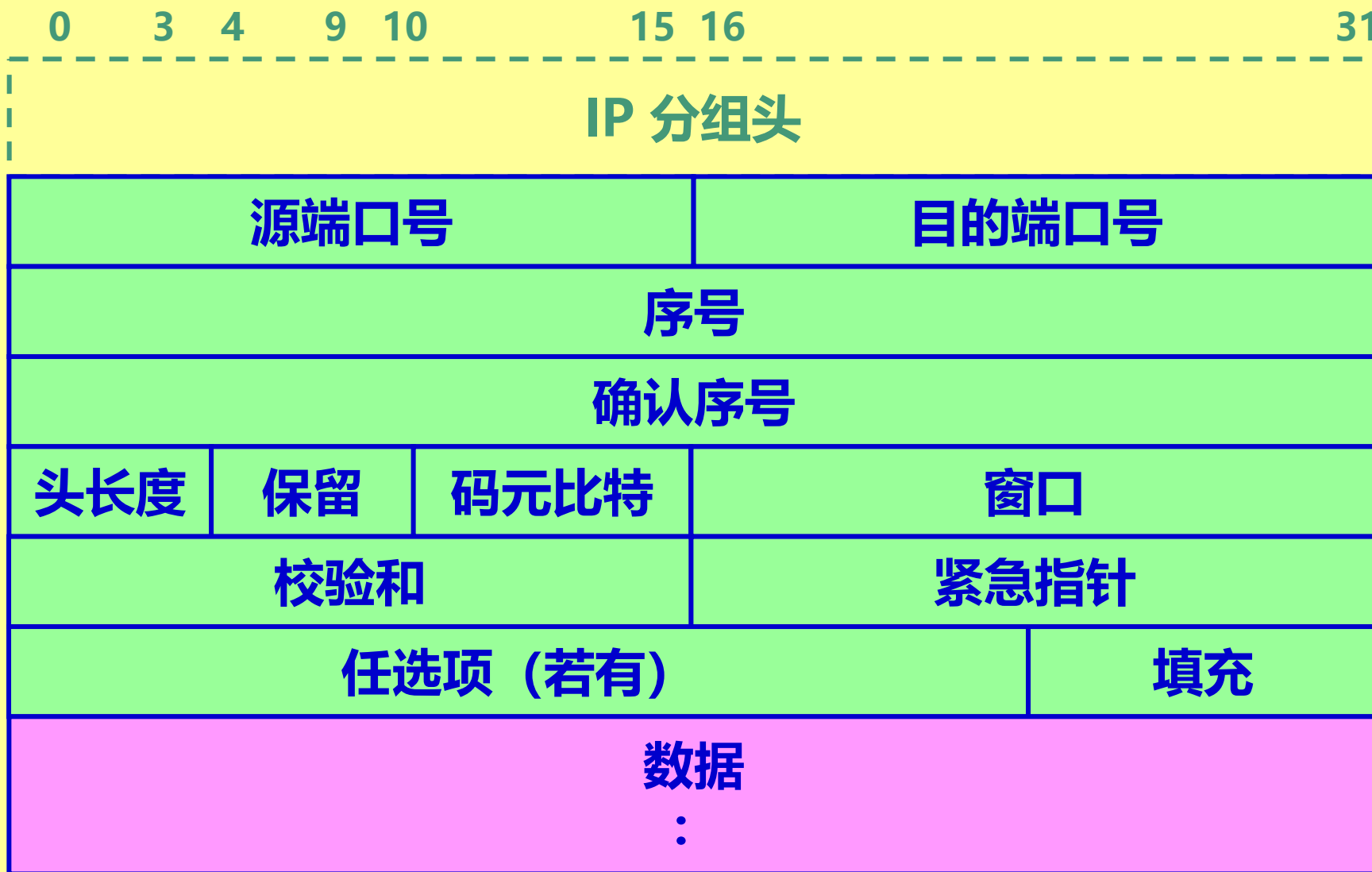
- ✓ 报文段分为头部和数据区，并封装在一个 IP 分组中传输



❖ TCP 头：携带必须的标识和控制信息，包括：

- ✓ 连接标识：
 - ✉ 源端口和目的端口：标识连接的两个端点
- ✓ 差错和流量控制：
 - ✉ 序号：指出本报文段在发送方的数据字节流中的位置
 - ✉ 确认序号：指出本机希望接收的下一个字节的序号
- ✓

TCP 报文段的格式



TCP 数据流和报文段

❖ TCP 提供的传输服务是面向数据流的

- ✓ 数据流无结构
- ✓ 源端进程发送的数据以字节流的形式传输到目的进程

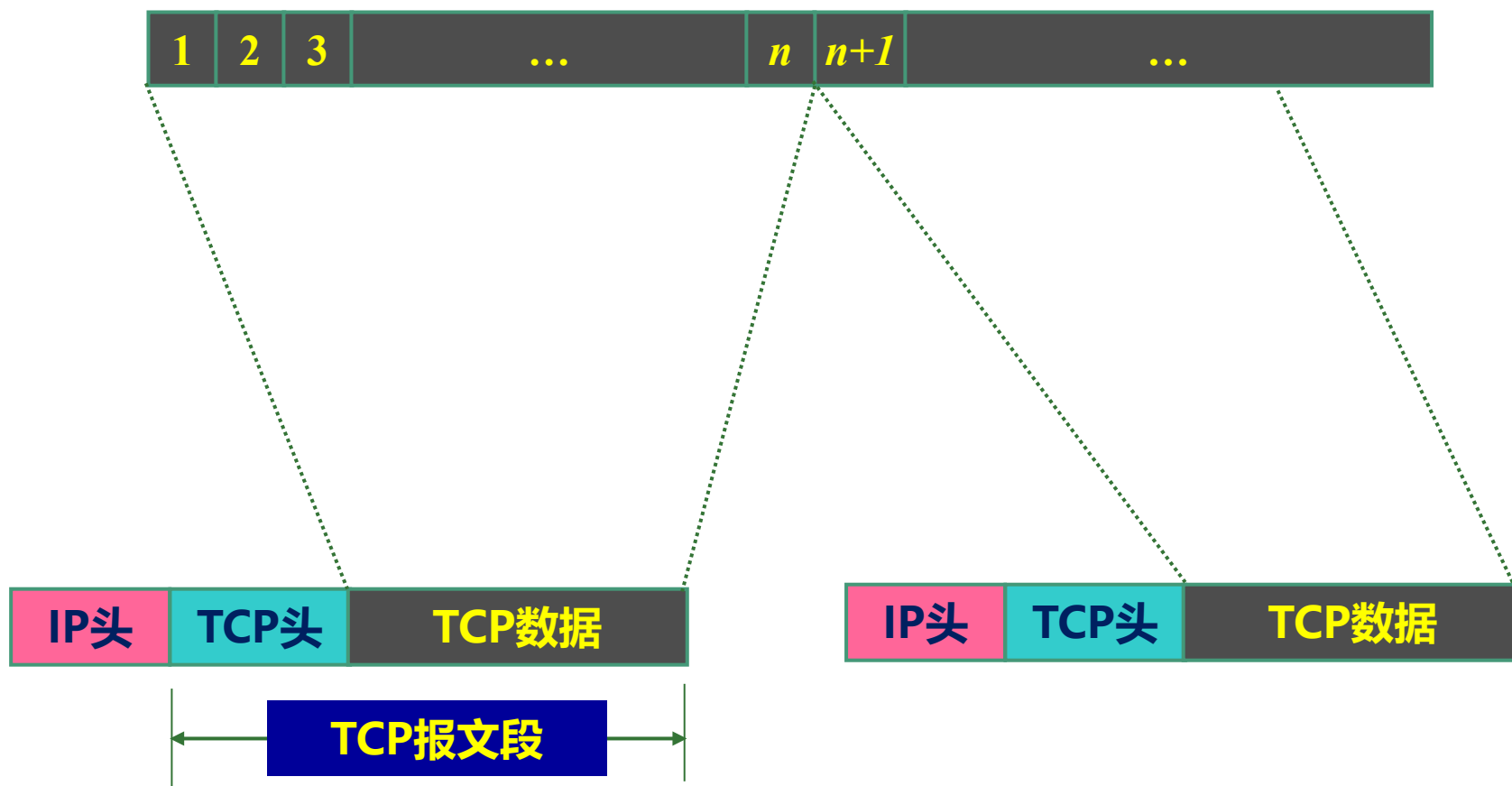
❖ 报文段 (segment) 的划分

- ✓ 为了便于传输, TCP 把一个字节流序列划分成若干个段
- ✓ 报文段是不定长的
- ✓ 一般, 每个段被封装在一个 IP 分组中传输
- ✓ 被封装的报文段存在以下几种情况:
 - ✉ 用于传输数据的报文段
 - ✉ 仅携带了确认信息的报文段
 - ✉ 携带连接建立请求或连接释放请求的报文段

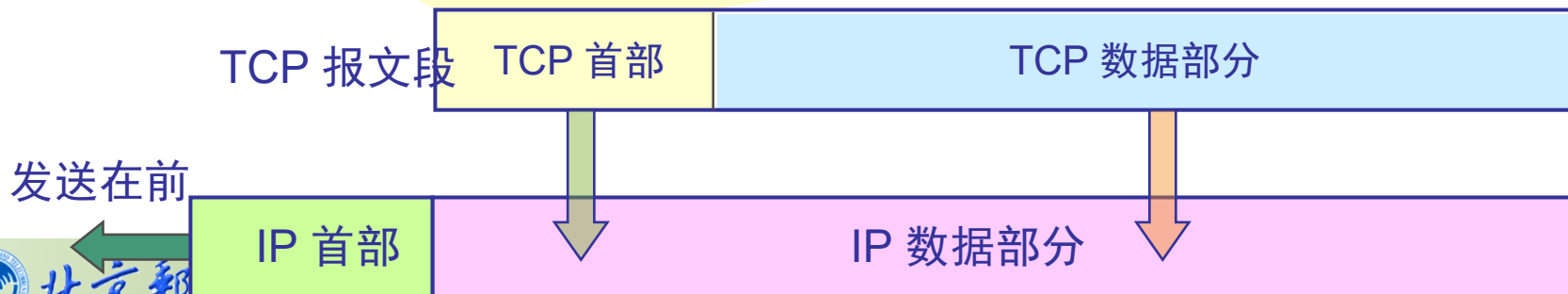
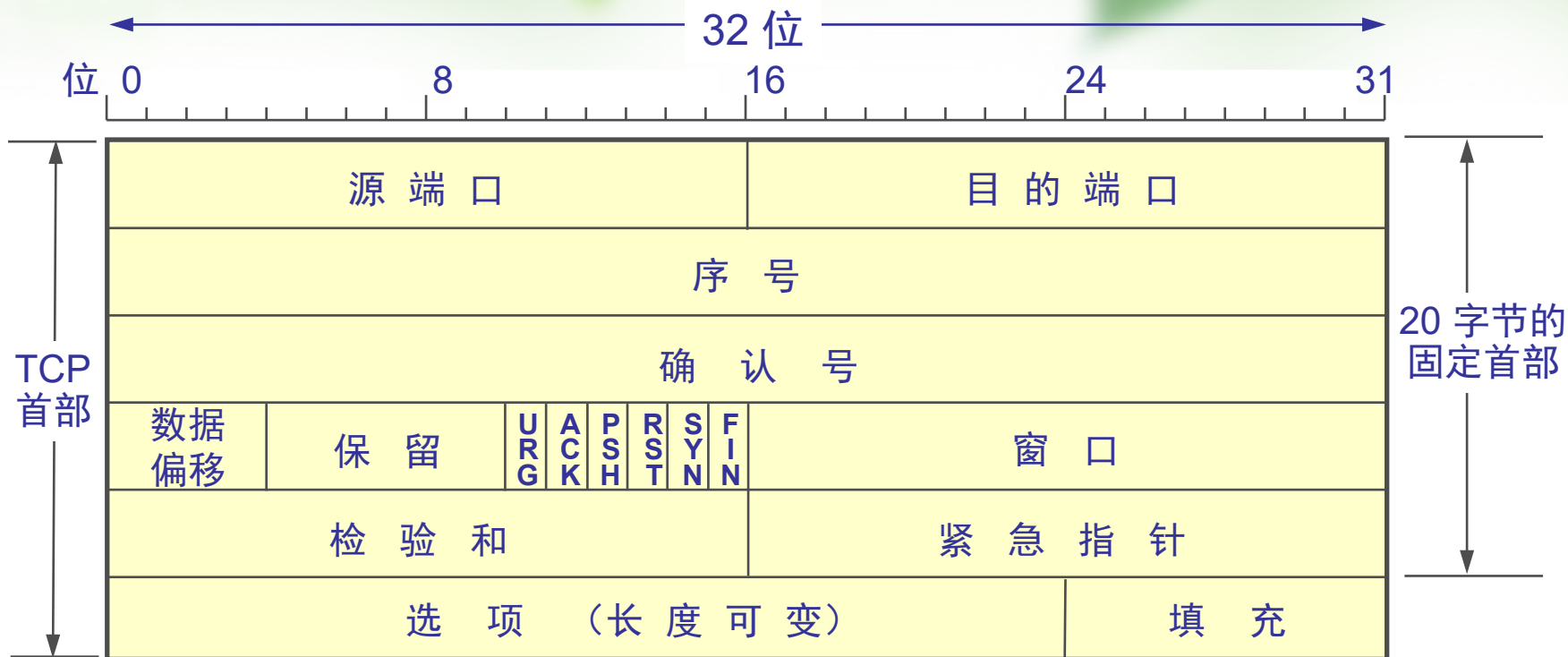


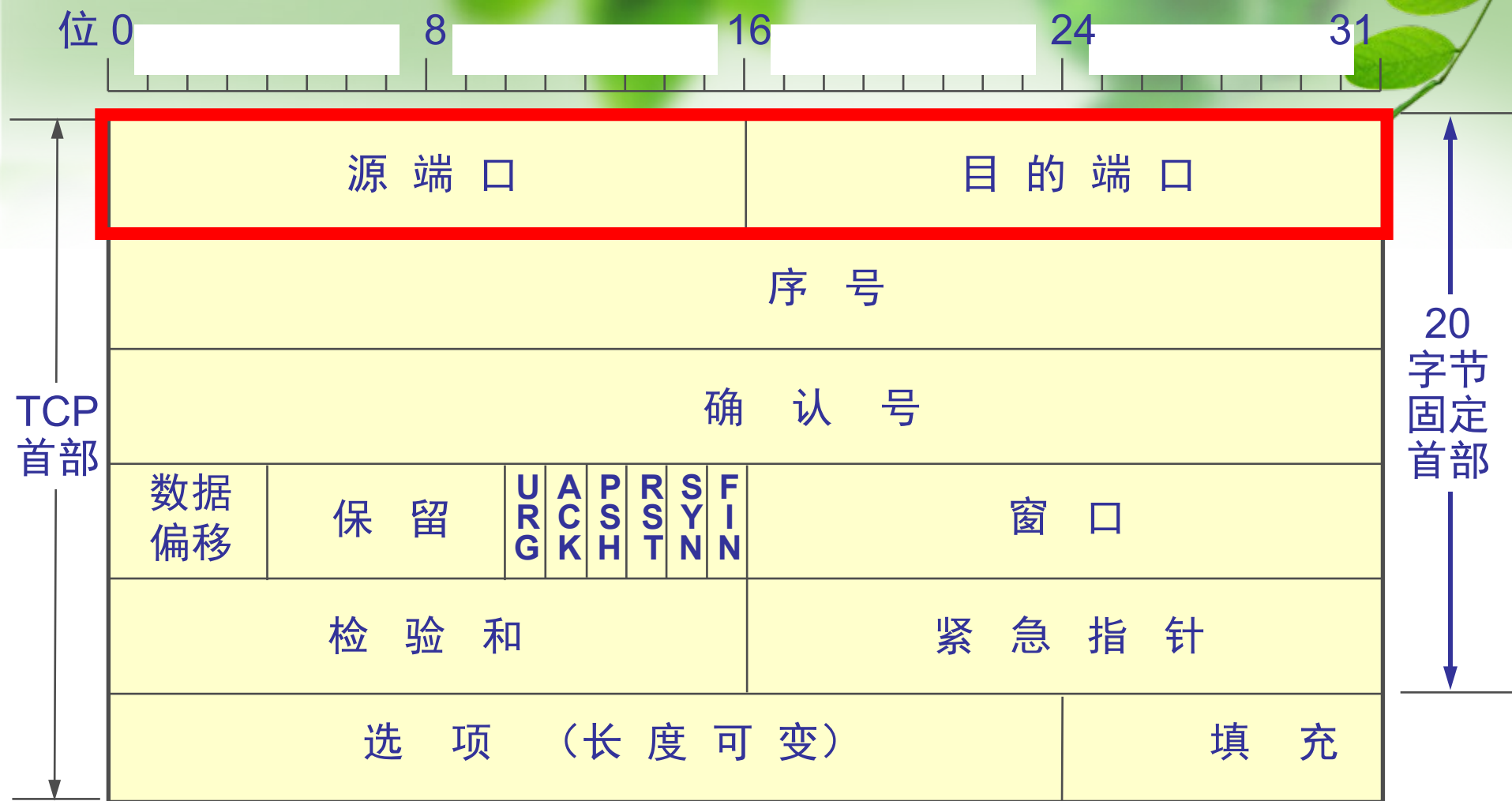
TCP 数据流和报文段

字节流...



TCP 报文段的首部格式





源端口和目的端口字段——各占 2 字节。端口是传输层与应用层的服务接口。传输层的复用和分用功能都要通过端口才能实现。



序号字段——占 4 字节。TCP 连接中传送的数据流中的每一个字节都编上一个序号。序号字段的值则指的是本报文段所发送的数据的第一个字节的序号。



确认号字段——占 4 字节，是期望收到对方的下一个报文段的数据的第一个字节的序号。



数据偏移（即首部长度的）——占 4 位，它指出 TCP 报文段的数据起始处距离 TCP 报文段的起始处有多远。“数据偏移”的单位是 32 位字（以 4 字节为计算单位）。

位 0 8 16 24 31

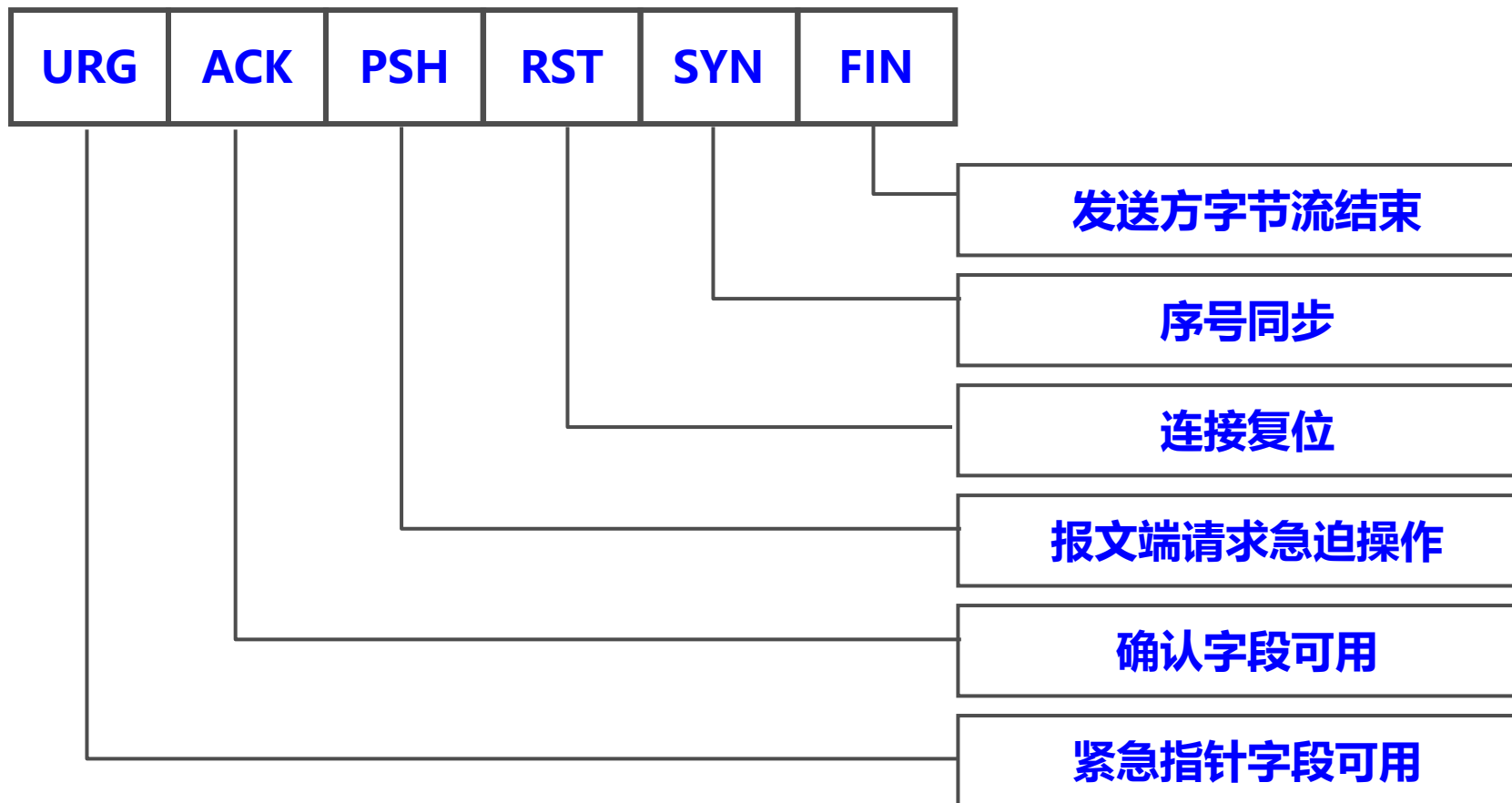


保留字段——占 6 位，保留为今后使用，但目前应置为 0。

TCP 报文段中的码元比特

❖ 码元比特字段 (CODE BITS) 6bit

✓ 指出报文段的目的和内容，给出报文头中其他字段的解释





紧急 URG —— 当 $URG = 1$ 时，表明紧急指针字段有效。它告诉系统此报文段中有紧急数据，应尽快传送(相当于高优先级的数据)。

位 0 8 16 24 31



确认 ACK —— 只有当 $ACK = 1$ 时确认号字段才有效。当 $ACK = 0$ 时，确认号无效。



推送 PSH (PuSH) —— 接收 TCP 收到 PSH = 1 的报文段，就尽快地交付接收应用进程，而不再等到整个缓存都填满了后再向上交付。



复位 RST (ReSeT) —— 当 $RST = 1$ 时，表明 TCP 连接中出现严重差错（如由于主机崩溃或其他原因），必须释放连接，然后再重新建立传输连接。





同步 SYN —— 同步 SYN = 1 表示这是一个连接请求或连接接受报文。



终止 FIN (FINis) —— 用来释放一个连接。FIN = 1 表明此报文段的发送端的数据已发送完毕，并要求释放传输连接。

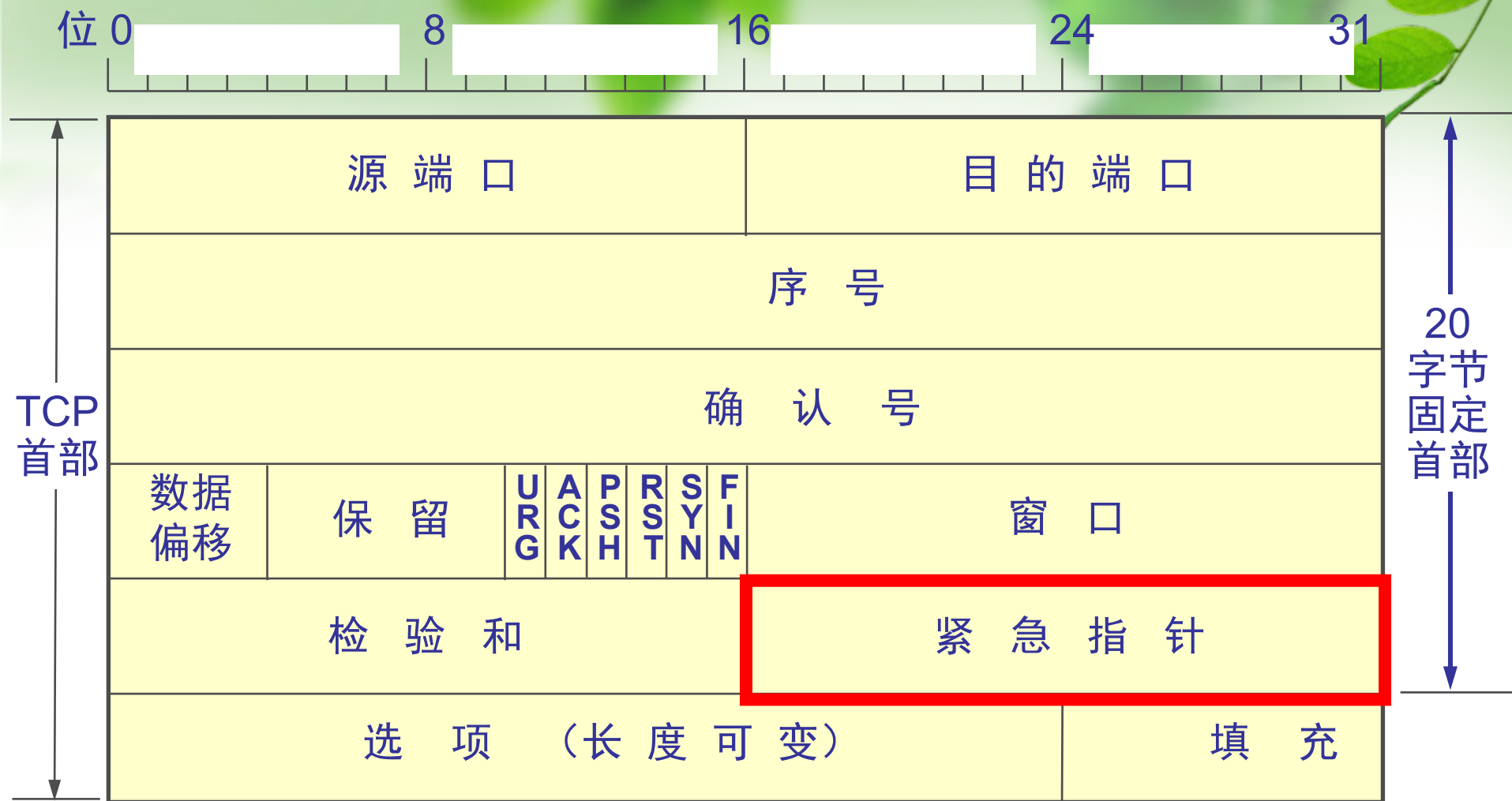
位 0 8 16 24 31



窗口字段 —— 占 2 字节，用来让对方设置发送窗口的依据，单位为字节。



检验和 —— 占 2 字节。检验和字段检验的范围包括首部和数据这两部分。在计算检验和时，要在 TCP 报文段的前面加上 12 字节的伪首部。



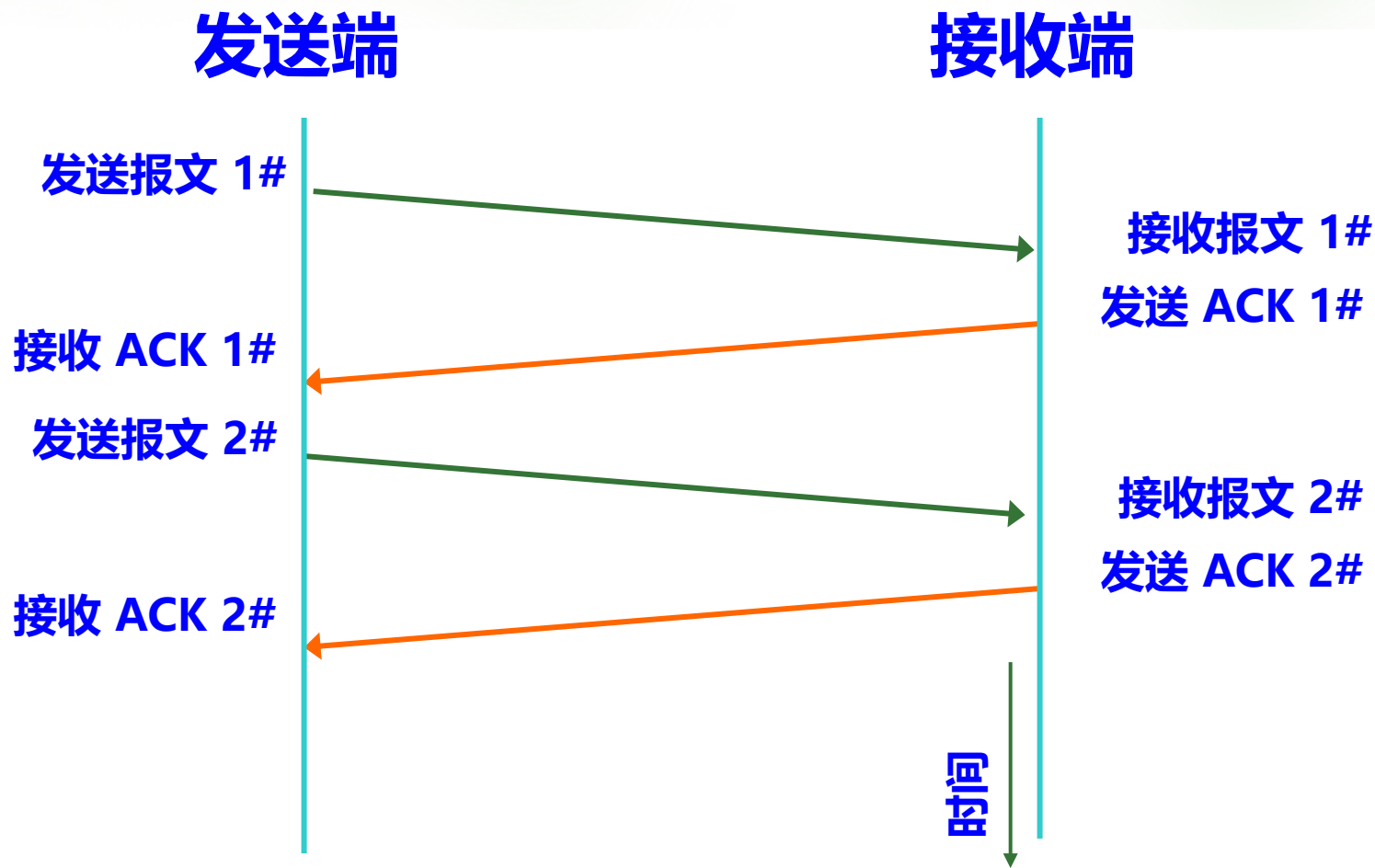
紧急指针字段 —— 占 16 位，指出在本报文段中紧急数据共有多少个字节（紧急数据放在本报文段数据的最前面）。

TCP 的可靠性机制——确认和重传

❖ TCP 传输服务的可靠性采用确认和重传机制来保证：

- ✓ 带重传的肯定确认技术作为 TCP 提供可靠性的基础
- ✓ TCP 要求连接的接收端在正确收到数据以后，向源端发送肯定确认信息 (ACK)
- ✓ 收到确认信息表明接收端已经正确接收到了数据
- ✓ 发送方在发送下一个报文段之前需要等待前一个报文段的确认信息
- ✓ 发送方为每一个发出的报文段都保存一个备份，
- ✓ 发送端发送一个报文段后立刻启动一个定时器；若在定时器超时的时候，远端仍未接收到目的端的数据应答，则认为前一个报文段传送失败，需要进行数据重传

典型的确认/重传机制



TCP 中的确认与重传机制

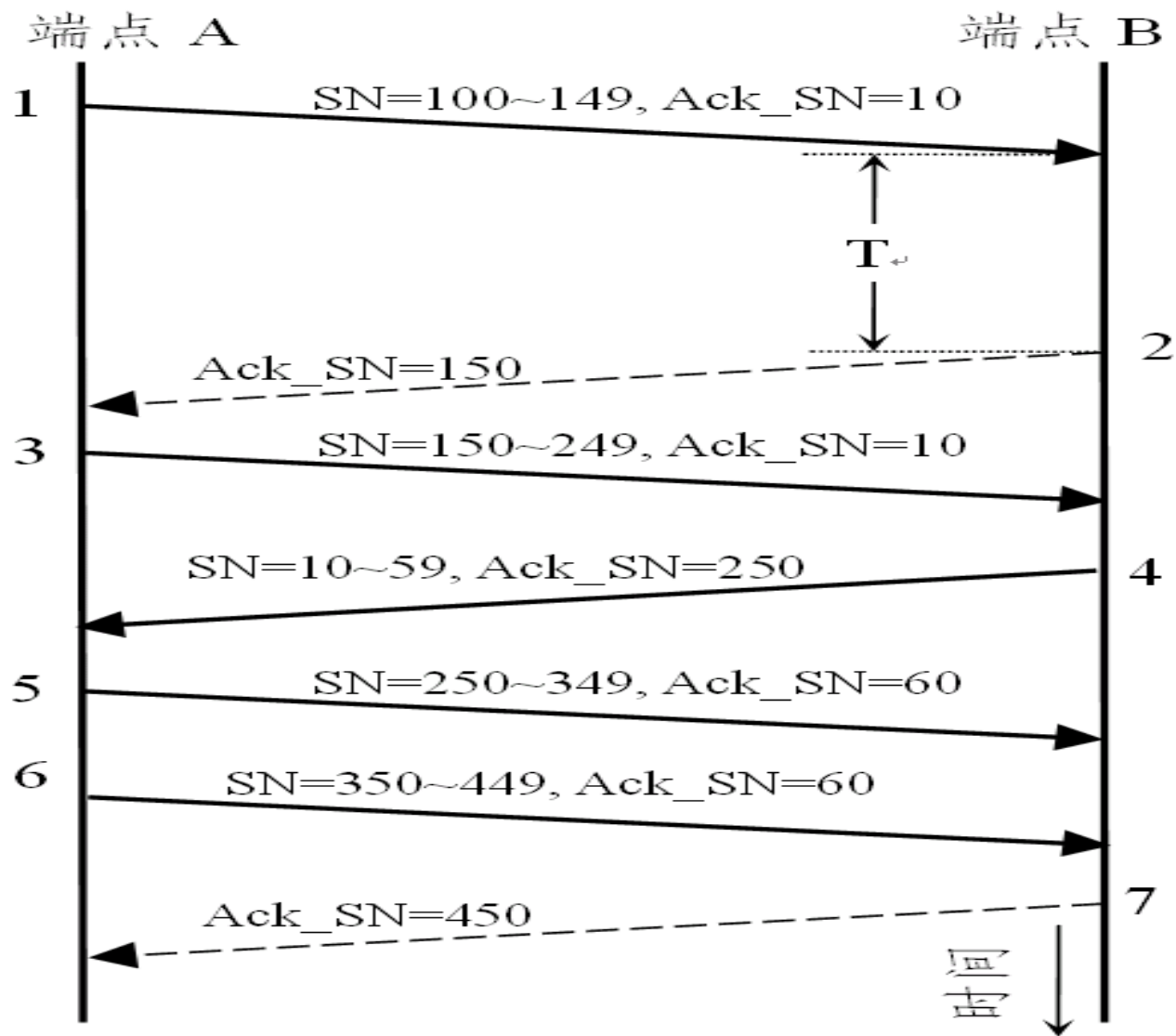
❖ TCP 流是无结构的字节流，并被划分为报文段

❖ TCP 中确认机制采用的是“累积确认”

- ✓ TCP 确认是针对数据流中的字节的，而不是针对报文段；其中使用的序号是特定字节在数据流中的序号（位置）
- ✓ 接收端确认的内容是：已经正确收到的、连续数据流中的最后一个字节（prefix of stream）
- ✓ 确认的方法是：接收端在确认中给出一个序号，其值为其最后收到的连续正确字节的序号加 1；实际上，确认信息中指出了接收端所希望接收到的下一个字节的序号
- ✓ 这种确认方法叫做累积确认，即报告接收端已经累积正确收到了数据流中的多少字节
- ✓ 确认信息利用 TCP 报文段头中的“确认序号”字段携带



TCP 确认



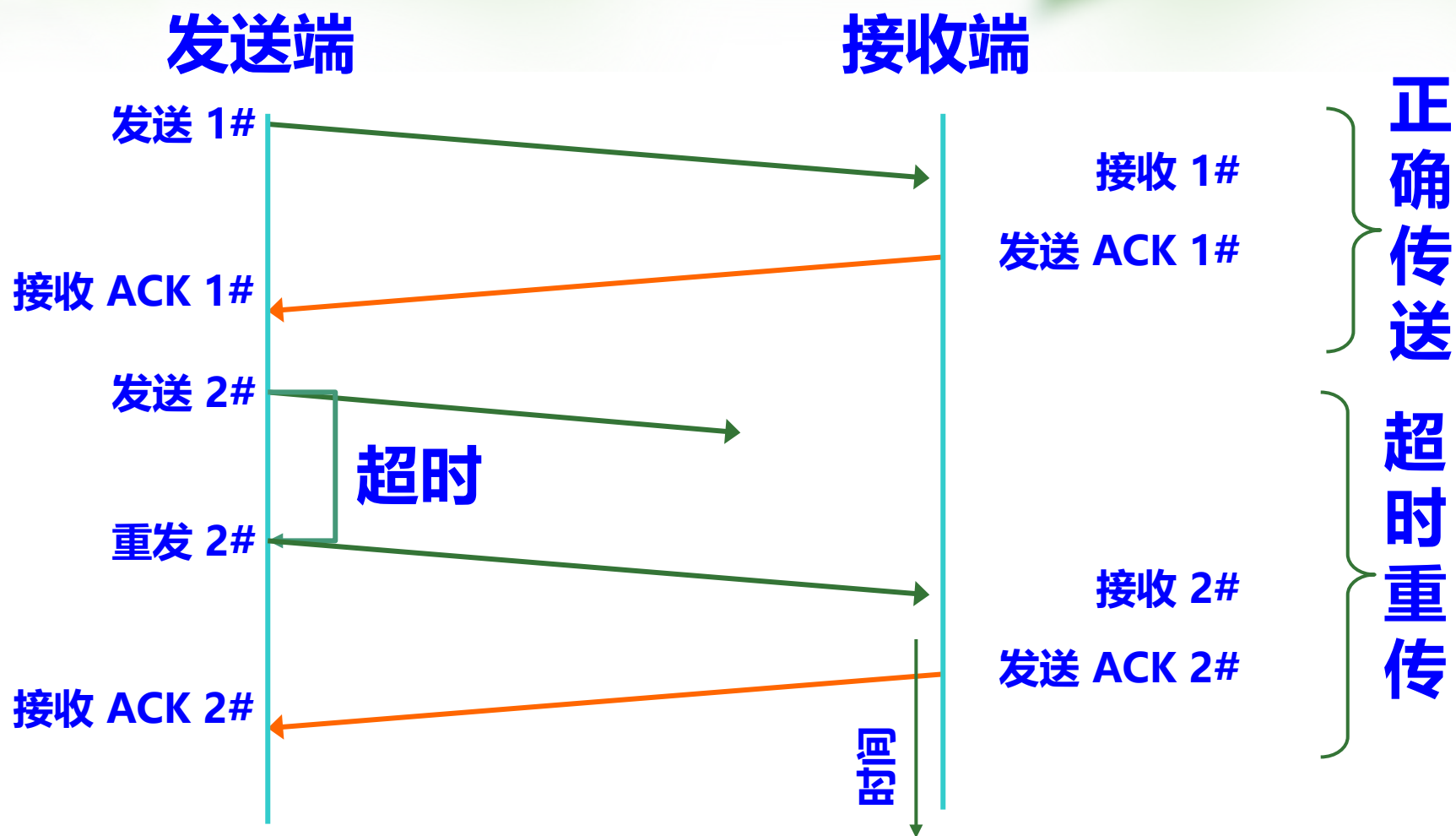
TCP 的确认

超时与重传

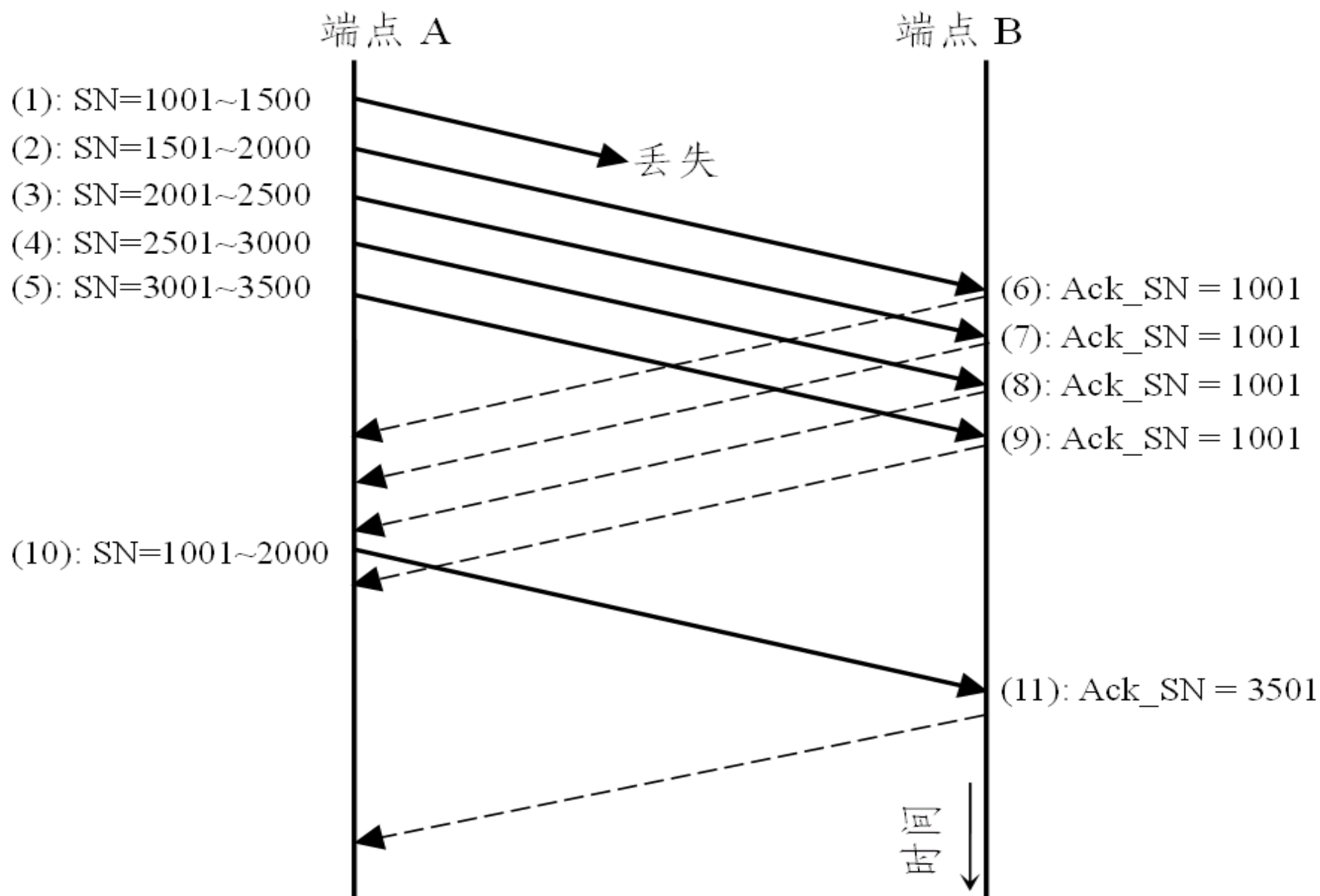
❖ TCP 采用超时重传技术来检测和处理报文段的丢失

- ✓ 源端每发送一个报文段并开始等待确认信息时，TCP 就启动一个定时器
- ✓ 如果在报文段数据的确认信息到达之前，定时器超时，则TCP 认为该报文段已经丢失或被破坏，然后重传这一报文段
- ✓ TCP 使用自适应重传算法以适应互连网络时延的变化：
 - ✉ TCP 监视每条连接的性能（网络时延），并由此推算出每个连接合适的定时器时间值

基本的超时重传



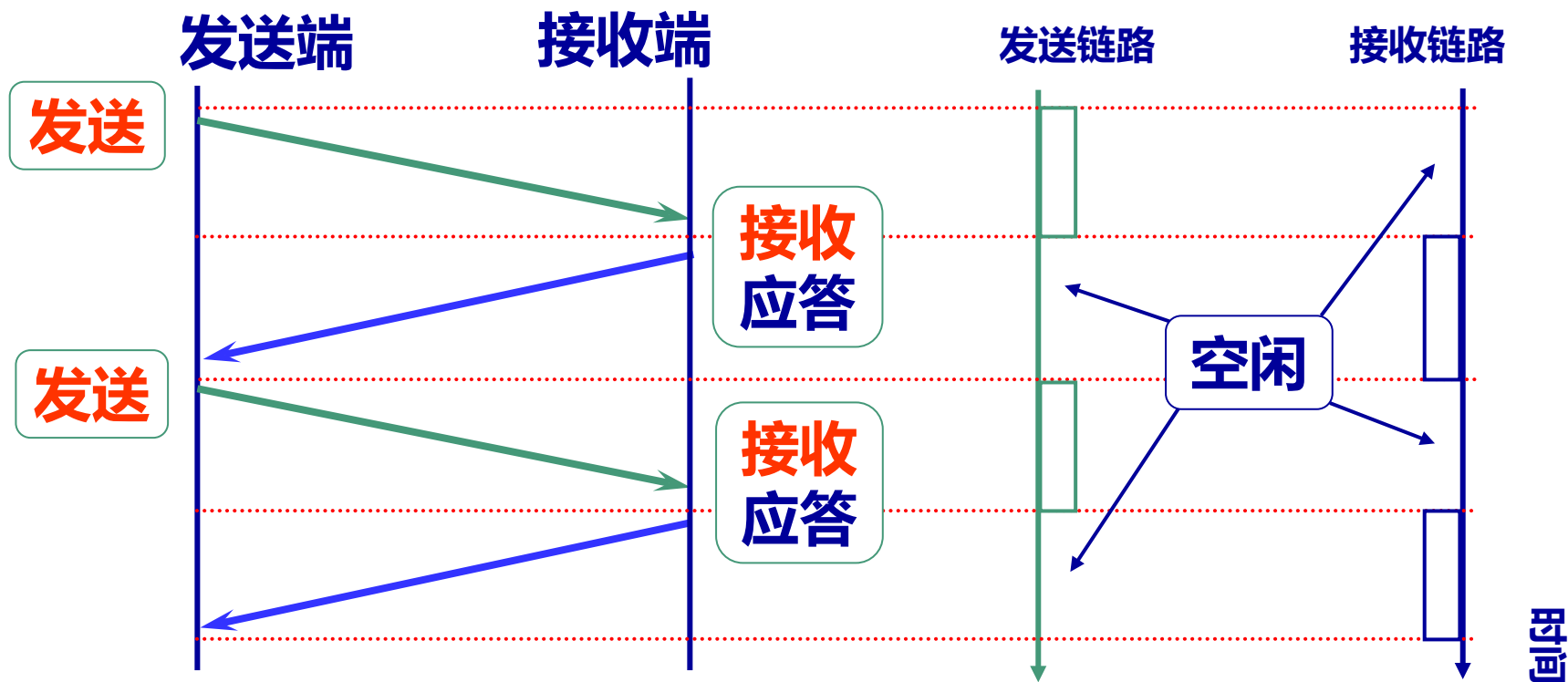
丢失的快速发现与重传



滑动窗口的引入

❖ 简单停等协议的缺陷：资源利用率不高

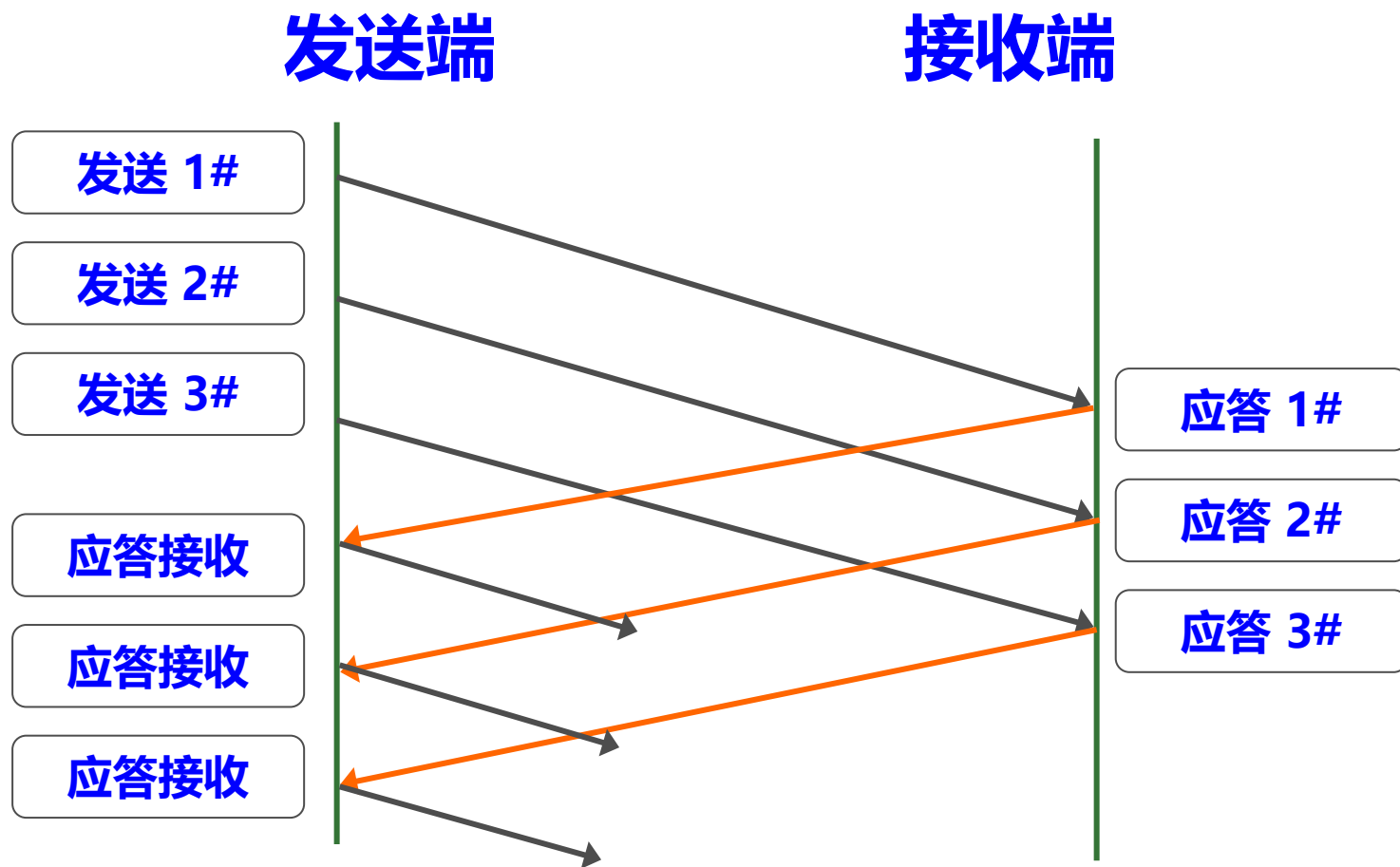
- ✓ 在接到确认信息前，必须推迟下一个报文段的发送
- ✓ 网络具有双向通信能力，但停等协议只允许数据单向传输
- ✓ 在等待响应的过程中网络完全空闲（特别在大时延网络）
- ✓ 如果在一个时延很大的网络上，这个问题就更突出



传输效率与滑动窗口

❖ 滑动窗口协议 —— 提高传输效率进行流量控制

✓ 允许发送方在确认信息到达前，发送多个报文段



传输效率与滑动窗口

- ❖ 发送窗口扩大时，允许发送端在未收到确认前发送更多的数据，具有更高数据传输率
- ❖ 当发送窗口的大小与传输通道的容量相匹配时，滑动窗口的效率达到最高

传输通道的容量 = 通道带宽 * 通道往返传输时延

- ❖ 例如：当通道带宽是 2048Kbps 时

✓ 若通道往返时延（RTT）等于 10ms 时，发送窗口应为 2560Byte，就可充分利用通路带宽

滑动窗口的工作过程

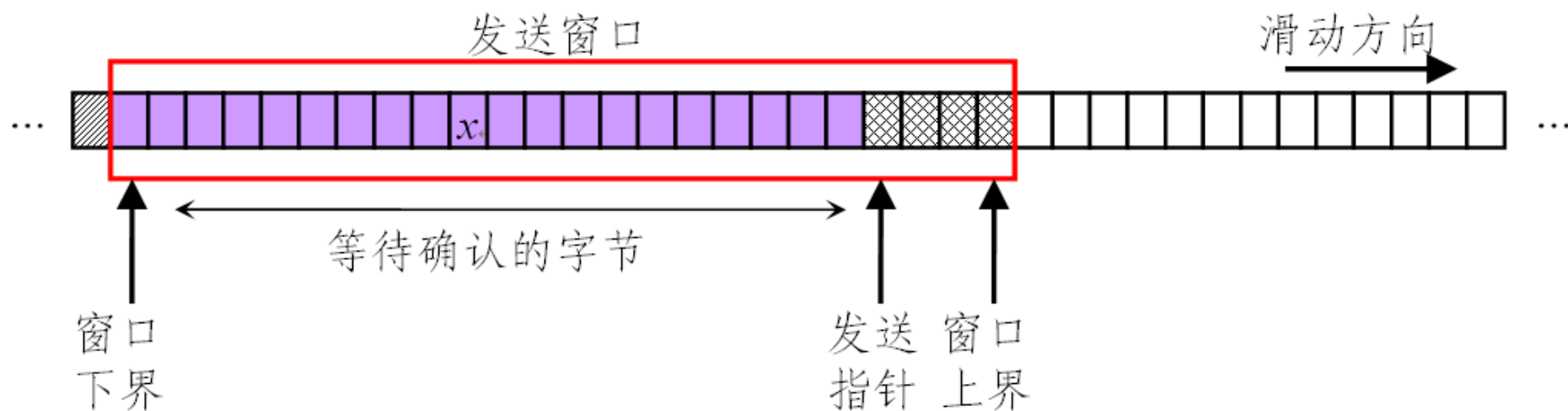
❖ 发送窗口

- ✓ 在未确认之前，允许发送的数据量由滑动窗口的大小确定
- ✓ 收到窗口下界的确认时，窗口就向前滑动
- ✓ 使新进入窗口的数据能够发送
- ✓ 滑动窗口只重传未被确认的数据

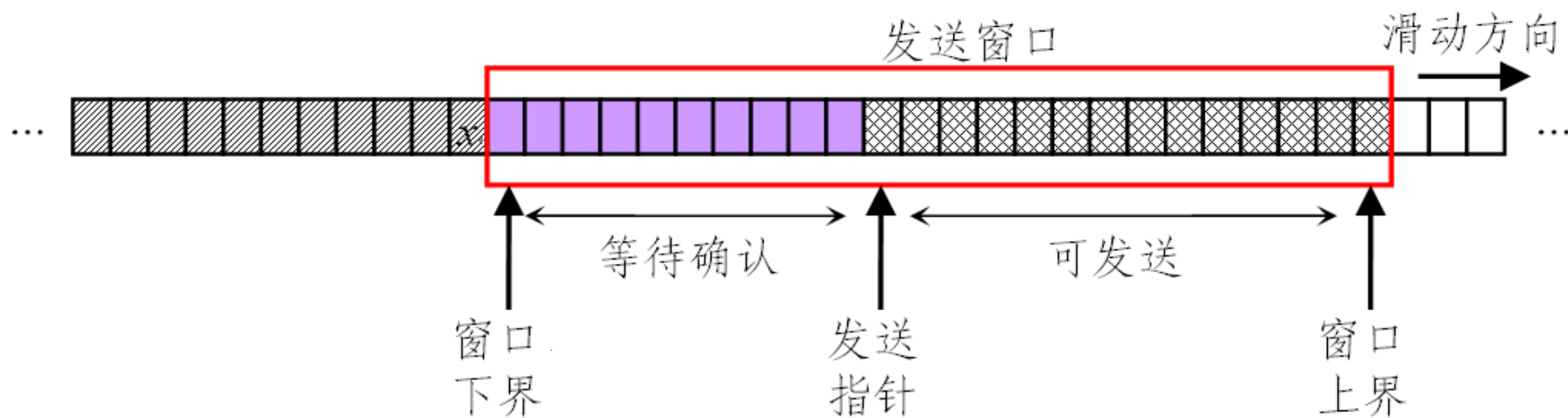
❖ 接收窗口

- ✓ 只接收窗口内到达的帧
- ✓ 窗口内下界的帧到达后，才发出确认（确认窗口下界）

TCP 滑动窗口

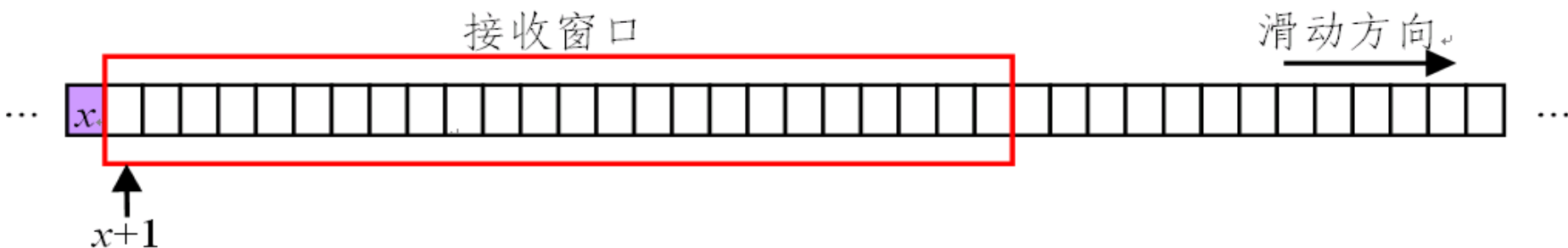


(a) 发送窗口的滑动前

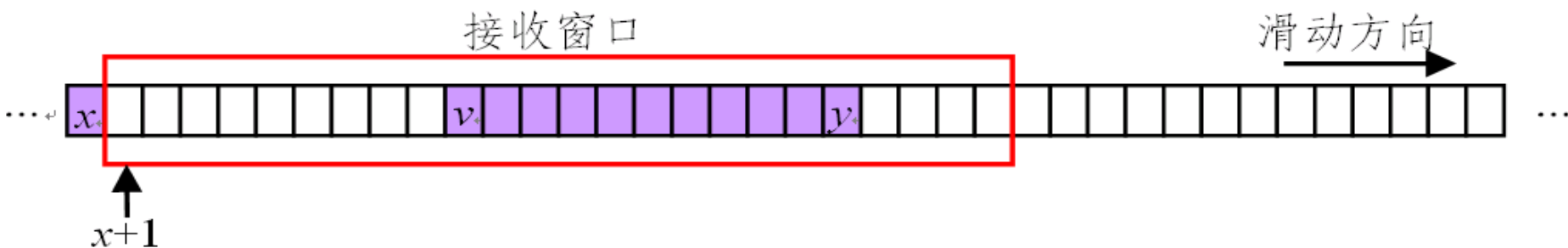


(b) 对某个字节 x 的确认到达，发送窗口的向前滑动

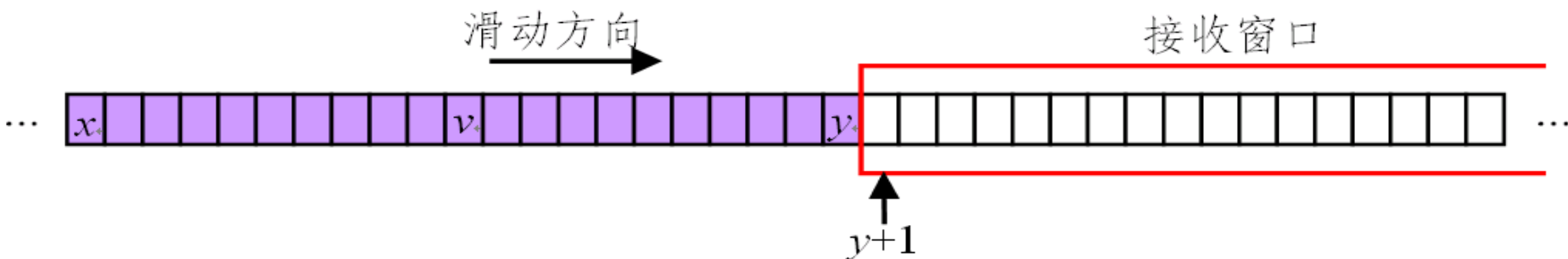
TCP 滑动窗口



(a) 接收窗口的初始情况



(b) 报文段 (字节 v 到 y) 到达后被缓存, 生成确认 $\text{Ack_SN} = x+1$



(c) 报文段 (字节 $x+1$ 到 $v-1$) 到达, 生成确认 $\text{Ack_SN} = y+1$, 接收窗口滑动到 $y+1$

TCP 的传输连接管理

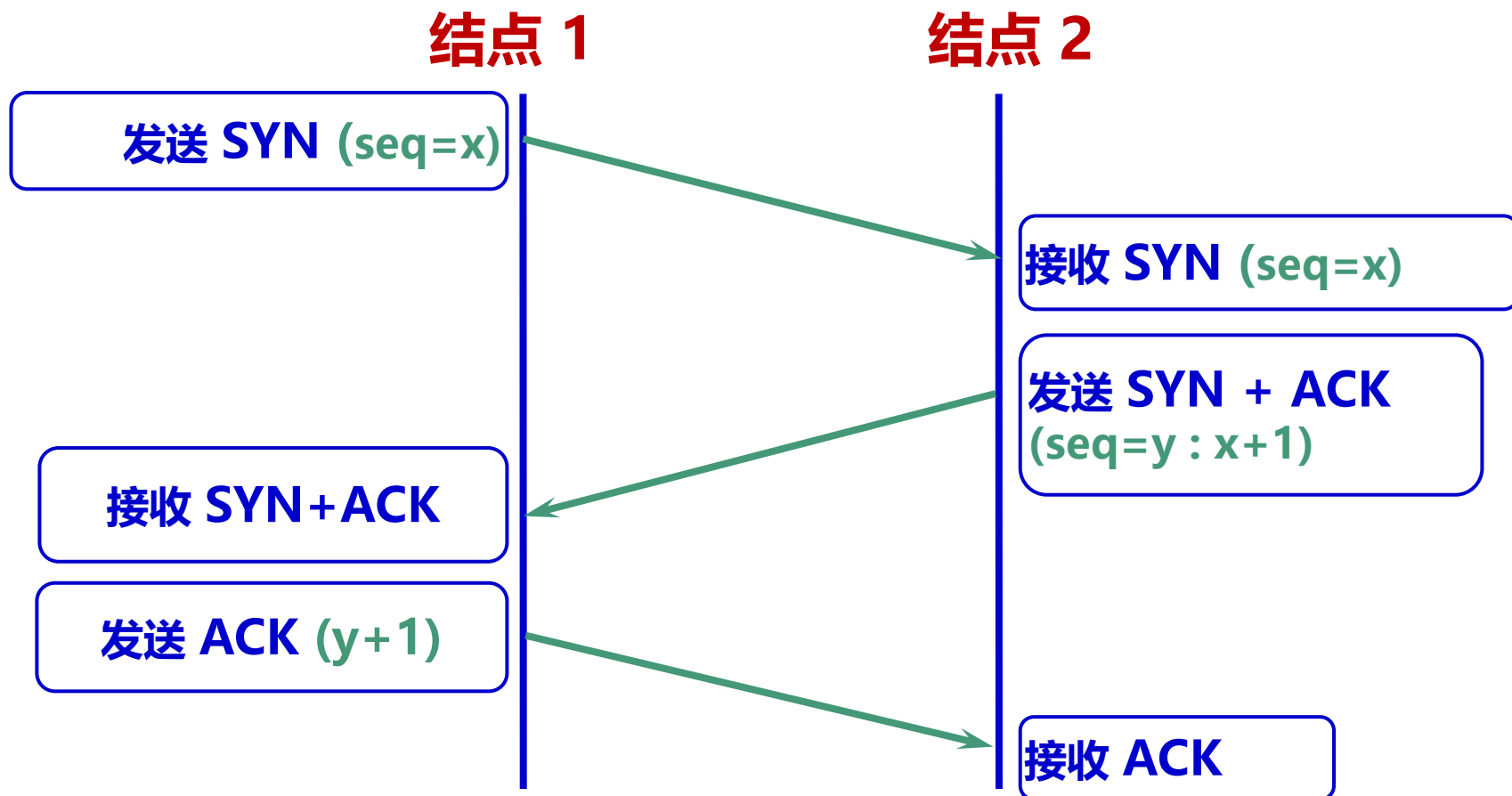
传输连接的三个阶段

- ❖ 传输连接就有三个阶段，即：连接建立、数据传送和连接释放。传输连接的管理就是使传输连接的建立和释放都能正常地进行。
- ❖ 连接建立过程中要解决以下三个问题：
 - ✓ 要使每一方能够确知对方的存在。
 - ✓ 要允许双方协商一些参数（如最大报文段长度，最大窗口大小，服务质量等）。
 - ✓ 能够对传输实体资源（如缓存大小，连接表中的项目等）进行分配。
- ❖ TCP 连接的建立都是采用客户服务器方式。
 - ✓ 主动发起连接建立的应用进程叫做客户(client)。
 - ✓ 被动等待连接建立的应用进程叫做服务器(server)。



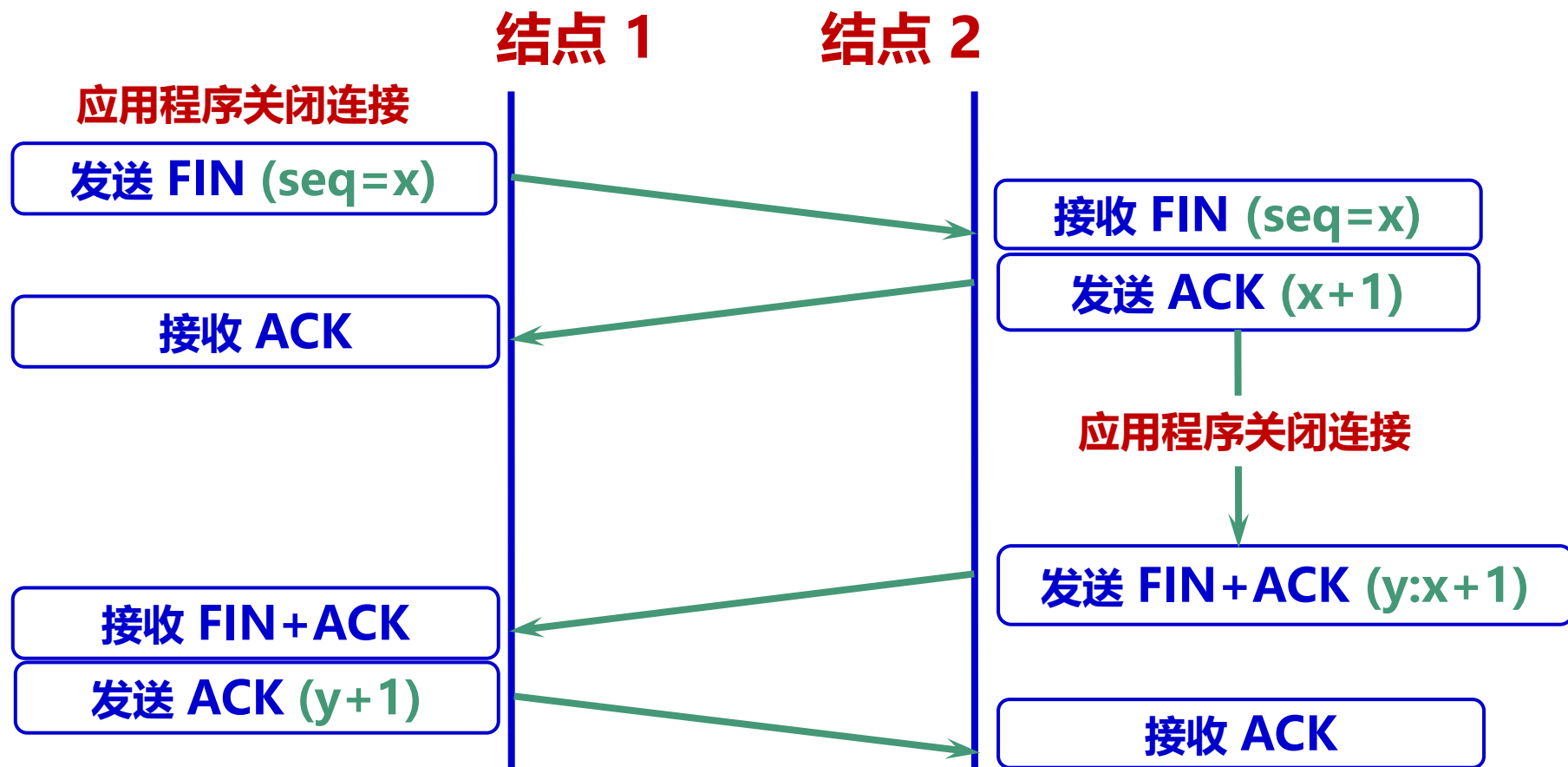
TCP 连接的建立

❖ TCP 使用三次握手协议来建立连接，三次握手协议是连接的两端正确同步的必要条件



TCP 连接的关闭

❖ 改进的三次握手协议来关闭 TCP 连接

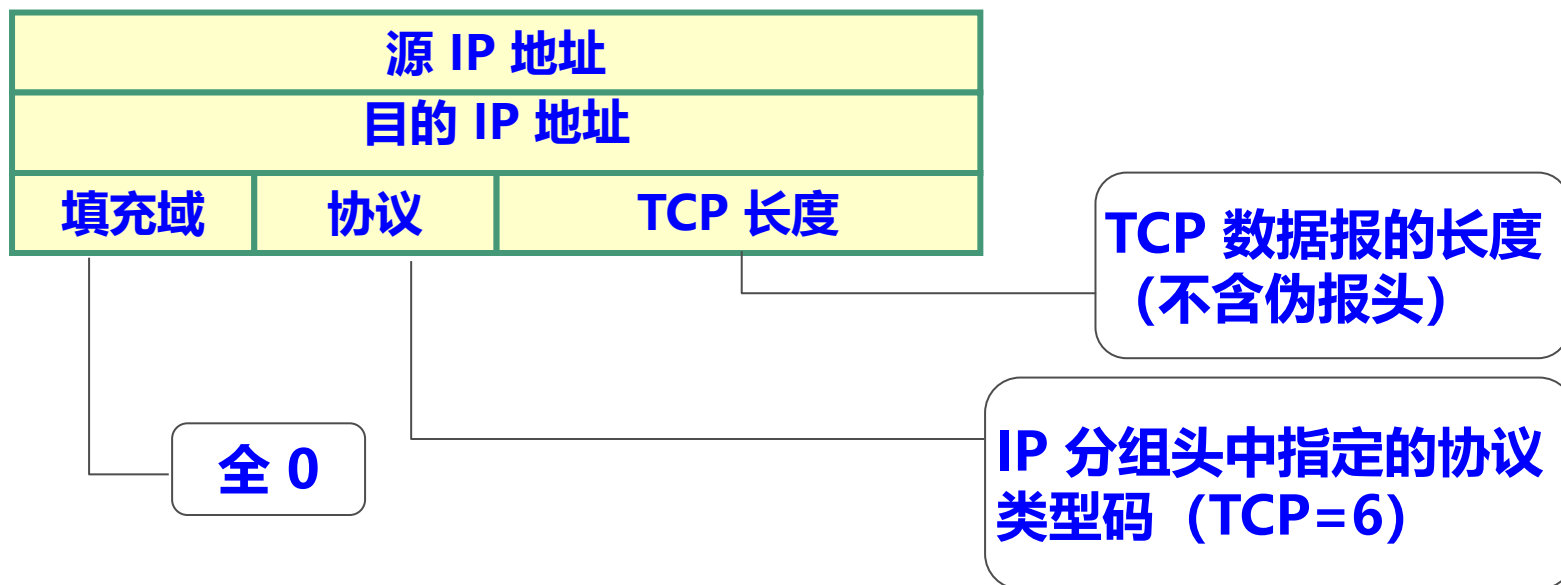


TCP 报文段的校验

❖ TCP 校验和的计算方法：同 IP 分组头的校验

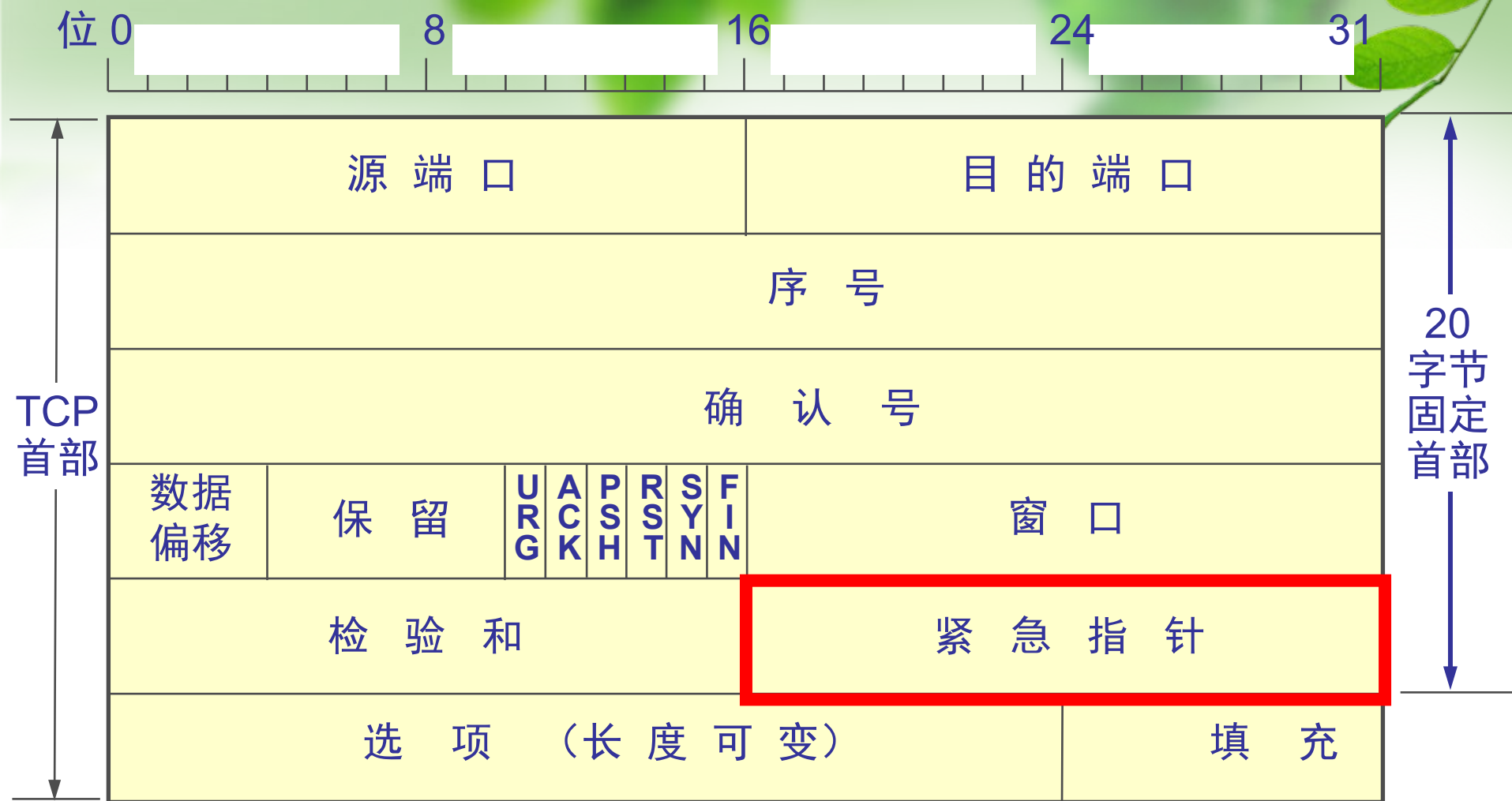
✓ 校验和计算：除覆盖数据报外，还覆盖一个 TCP 伪报头

✓ TCP 伪报头的目的与 UDP 基本相同





检验和 —— 占 2 字节。检验和字段检验的范围包括首部和数据这两部分。在计算检验和时，要在 TCP 报文段的前面加上 12 字节的伪首部。



紧急指针字段 —— 占 16 位，指出在本报文段中紧急数据共有多少个字节（紧急数据放在本报文段数据的最前面）。

位 0 8 16 24 31



窗口字段 —— 占 2 字节，用来让对方设置发送窗口的依据，单位为字节。

TCP 流量控制

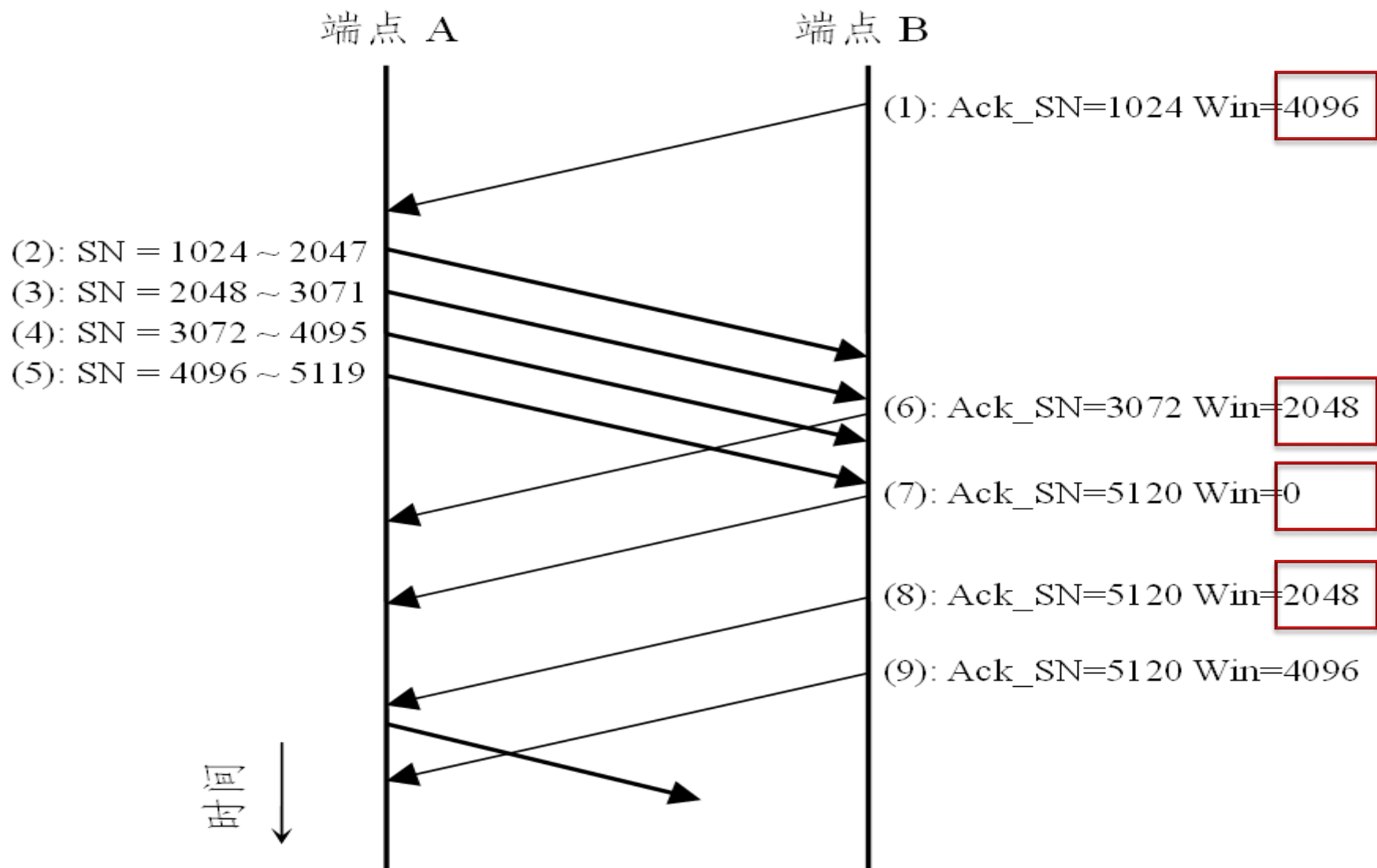
❖ TCP 通过可变的窗口大小来进行流量控制

- ✓ TCP 允许随时改变窗口大小
- ✓ 在确认报文中除确认序号（收到的字节）外，还包含窗口通告，说明接收方还可接收数据的能力
- ✓ 窗口通告值可被认为是当前接收缓冲区的大小
- ✓ 窗口通告值增加时，发送方可扩大其发送窗口的大小；窗口通告值减少时，发送方则应降低其发送窗口的大小

❖ 可变窗口的优点

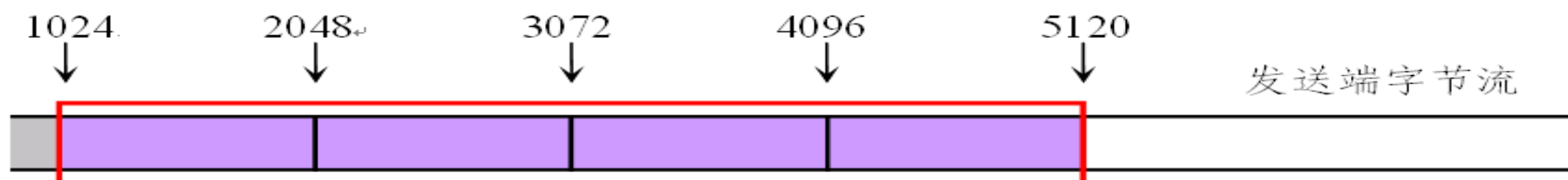
- ✓ 不仅提供可靠传输，而且还提供流量控制
- ✓ 当接收方缓冲区将要充满时，就可减小其窗口通告的值
- ✓ 在极端的情况下，接收方可使用零通告值来要求停止所有的传输

TCP 流量控制



TCP 的窗口通告的工作原理（交互过程）

TCP 流量控制



- (a) 接收到报文段 1 以后，发送窗口大小调整为 4096，并连续发送 4 个报文段，等待确认



- (b) 接收到报文段 6 以后，发送窗口的下界向前滑动，同时窗口大小调整为 2048，但不能发送更多数据



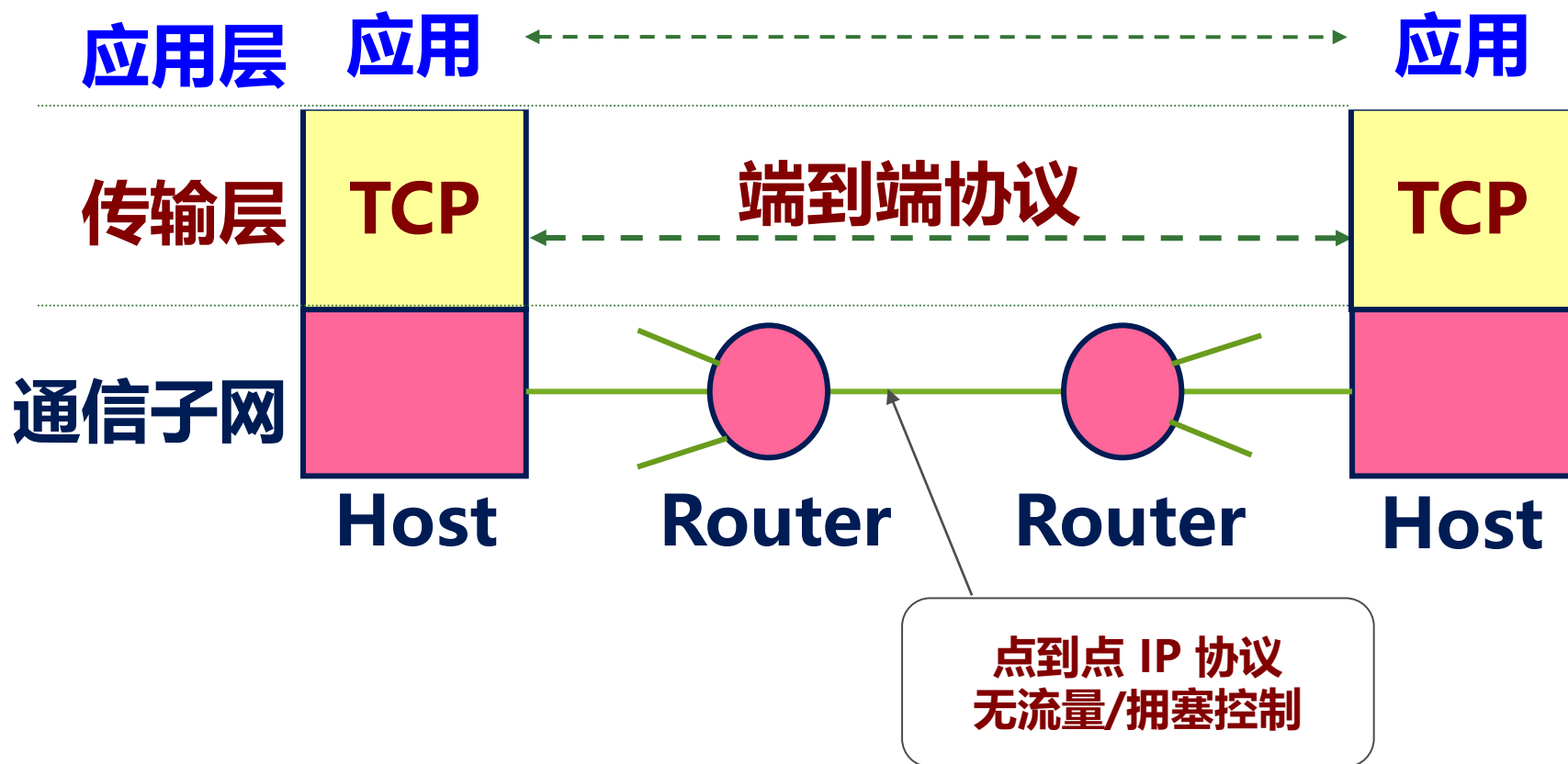
- (c) 接收到报文段 7 以后，发送窗口的下界向前滑动，但窗口大小为 0，将暂停发送



- (d) 接收到报文段 8 以后，窗口大调整为 2048 (窗口上界移动，下界不变)，可已发送新的数据

TCP 的拥塞控制

- ❖ TCP 是端到端的协议
- ❖ TCP 的拥塞控制是在端到端的基础上进行的



网络拥塞 (Congestion)

❖ 拥塞是所有分组网络所需面临的问题

- ✓ 分组网络，基于统计复用，允许资源竞争
- ✓ 进入网络的分组流可能具有很大突发性和不可预测性
- ✓ 短时间内大量分组竞争同一资源，将导致相应队列快速增长和溢出，使网络无法达到预定的性能和服务质量 (QoS)
- ✓ 这种情况就是网络拥塞
- ✓ 网络拥塞也可能由于网络内部的某些错误状态引起

❖ 拥塞表现为网络性能下降和用户 QoS 不能得到满足

- ✓ 排队的长度急剧增加，排队时延大大增加
- ✓ 很快超过队列容量，发生溢出并造成分组丢失
- ✓ 若同时在多个资源上产生拥塞，网络性能将明显下降

网络拥塞的危害

❖ 对于用户：用户预定的 QoS 不能得到满足

- ✓ 时间透明性：不可接受的端到端传输时延
- ✓ 语义透明性：大量的分组丢失

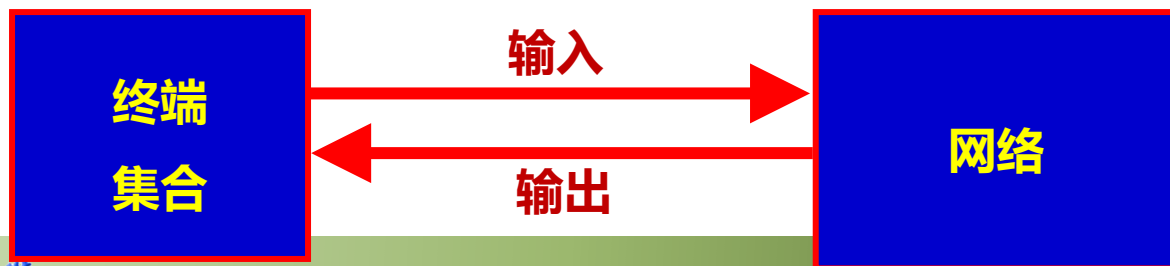
❖ 对于网络：

- ✓ 在拥塞发生处，所有连接的 QoS 将受到影响
- ✓ 若网络中多处同时出现拥塞，将严重影响网络性能
- ✓ 网络性能的描述：

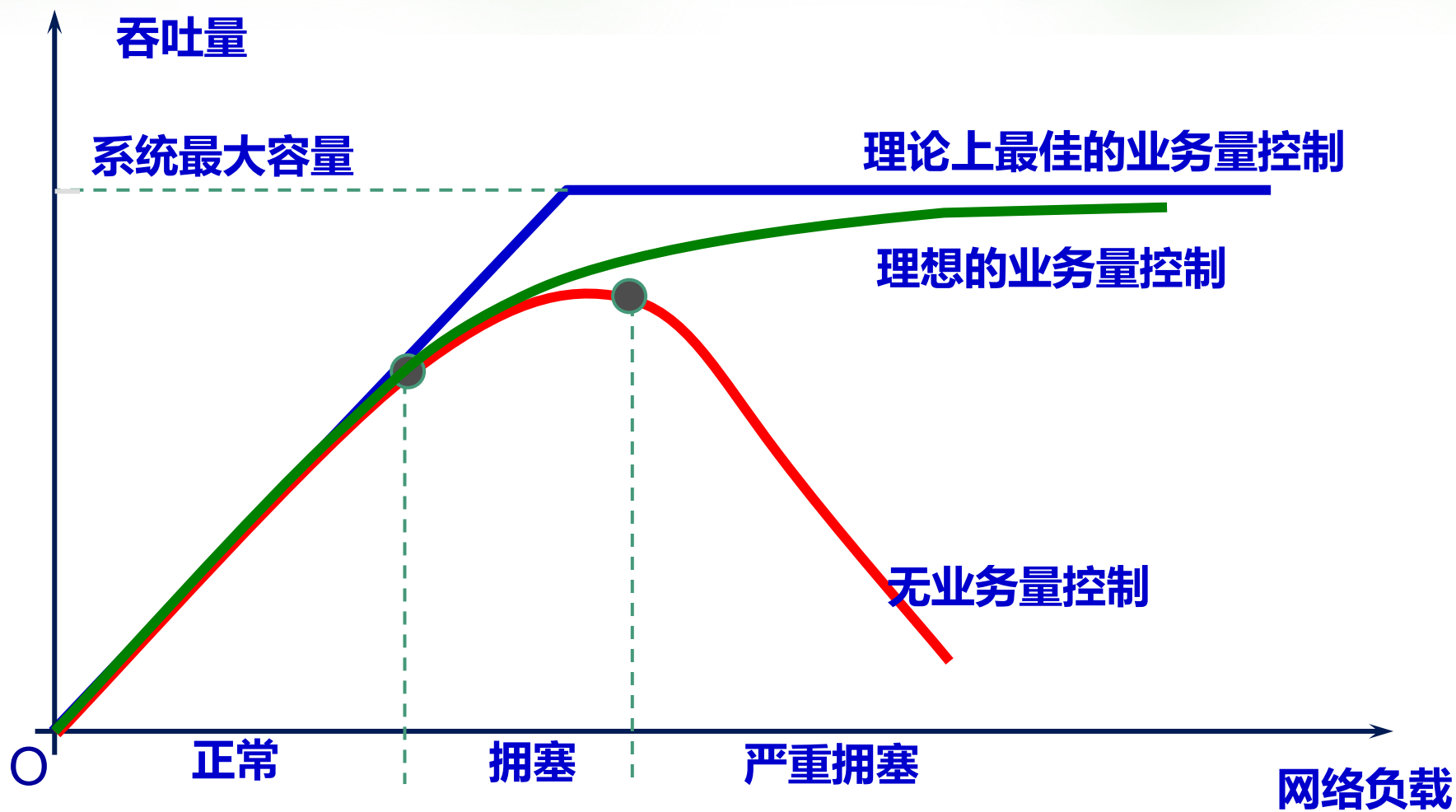
✉ 网络负载：所有终端向网络中注入的业务量速率的总和

✉ 网络吞吐量：业务量被网络有效传送到目的终端的速率的总和

✉ 网络容量：网络吞吐量的最大值

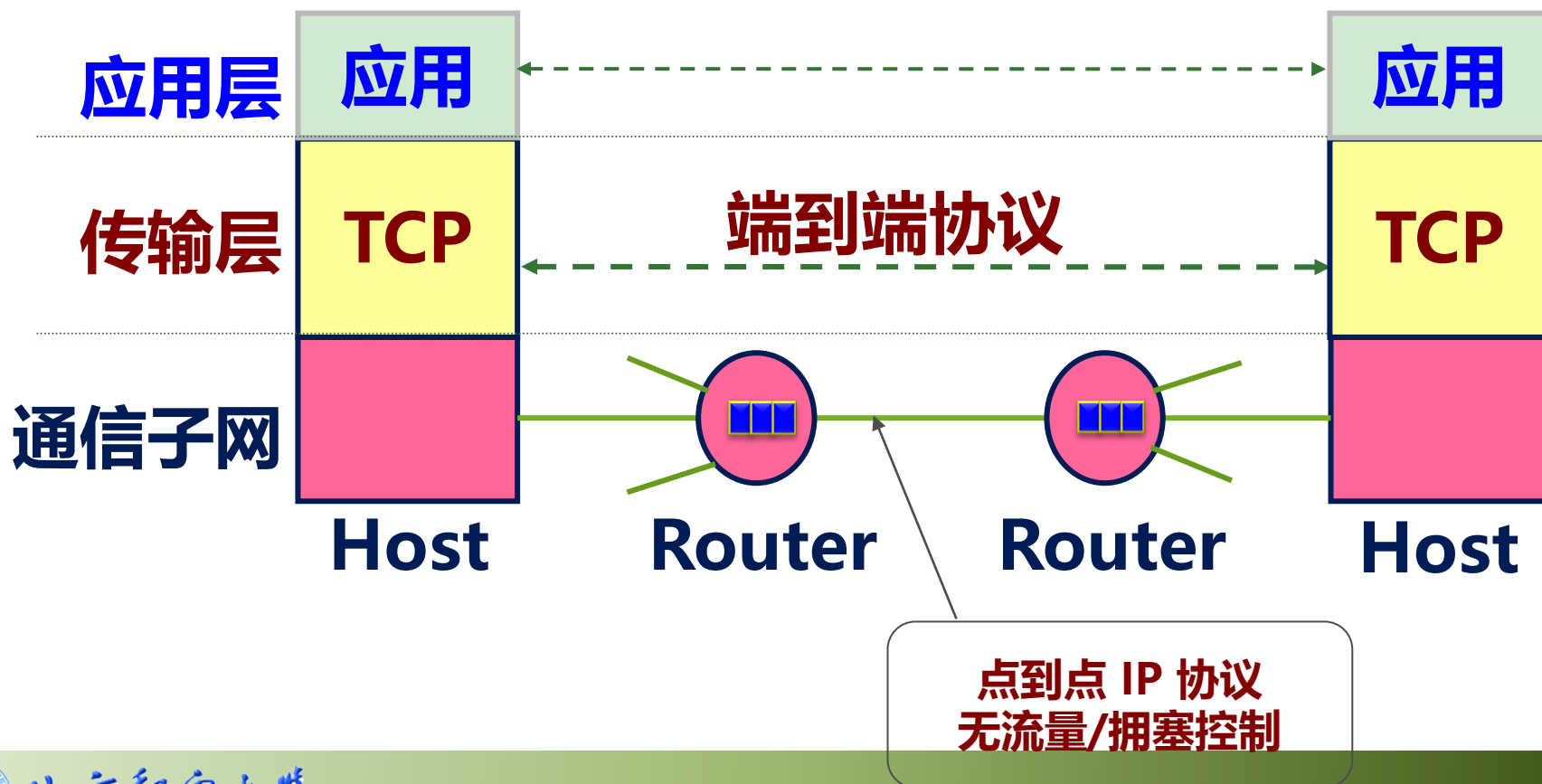


拥塞对网络性能的影响



TCP 的拥塞控制

- ❖ TCP 是端到端的协议
- ❖ TCP 的拥塞控制是在端到端的基础上进行的，降低发送速率



TCP 对拥塞的响应

❖ 在出现拥塞时，端点不能了解拥塞发生的细节：

- ✓ 发生拥塞的原因
- ✓ 拥塞发生的位置
- ✓ 对通信连接的端点来说，拥塞通常表现为通信时延的增加

❖ TCP 的超时重传机制可能导致拥塞的加剧

- ✓ TCP 协议采用超时重传机制：在通信时延增加时，其响应是不断重传报文段，这种重传将会加剧拥塞的程度。
- ✓ 在出现网络拥塞时，若对分组的重传不加抑制，数据流量增加又将进一步增加时延，出现的恶性循环，最终将导致整个网络完全失效；这种现象称为拥塞崩溃。

❖ 为避免拥塞崩溃，TCP 必须在拥塞发生时减少其数据的传输量

TCP 拥塞窗口

❖ TCP 的拥塞窗口 (Congestion Window)

✓ TCP 采用拥塞窗口机制来控制发送窗口的大小，防止拥塞的发生，及在拥塞出现时逐步消除拥塞

❖ 采用拥塞窗口机制后，在任何时刻，TCP 协议的发送窗口的大小将根据以下公式计算：

$$AWin = \min(Rcvr_Adv, Cong_Win)$$

✓ $AWin$: Allowed Window Size

✓ $Rcvr_Adv$: Receiver Advertisement Size

✓ $Cong_Win$: Congestion Window Size

拥塞时拥塞窗口的确定

❖ 为避免拥塞，拥塞窗口采用的加速递减策略

- ✓ TCP 假设大多数的报文丢失都是由于网络拥塞造成的
- ✓ 一旦发现报文段丢失（超时未确认），立即将拥塞窗口大小减半，最小可将拥塞窗口的大小减为 1
- ✓ 减少拥塞窗口大小的同时，对于所有保留在发送窗口中未确认的报文段，将其重传定时器的时限加倍，按照指数规律补偿重传定时器



拥塞恢复

❖ 在拥塞结束后，采用慢启动算法逐步恢复传输速率

- ✓ 慢启动算法用于新建连接的刚开始进行数据传输，或用于拥塞结束避免短时间内再次出现拥塞
- ✓ 慢启动算法中，拥塞窗口的初始值是一个报文段的长度
- ✓ 每当发送端接收到一个确认后，就可将拥塞窗口增加 1
- ✓ 慢启动可以防止连接新建或拥塞解除后流量的突发增加

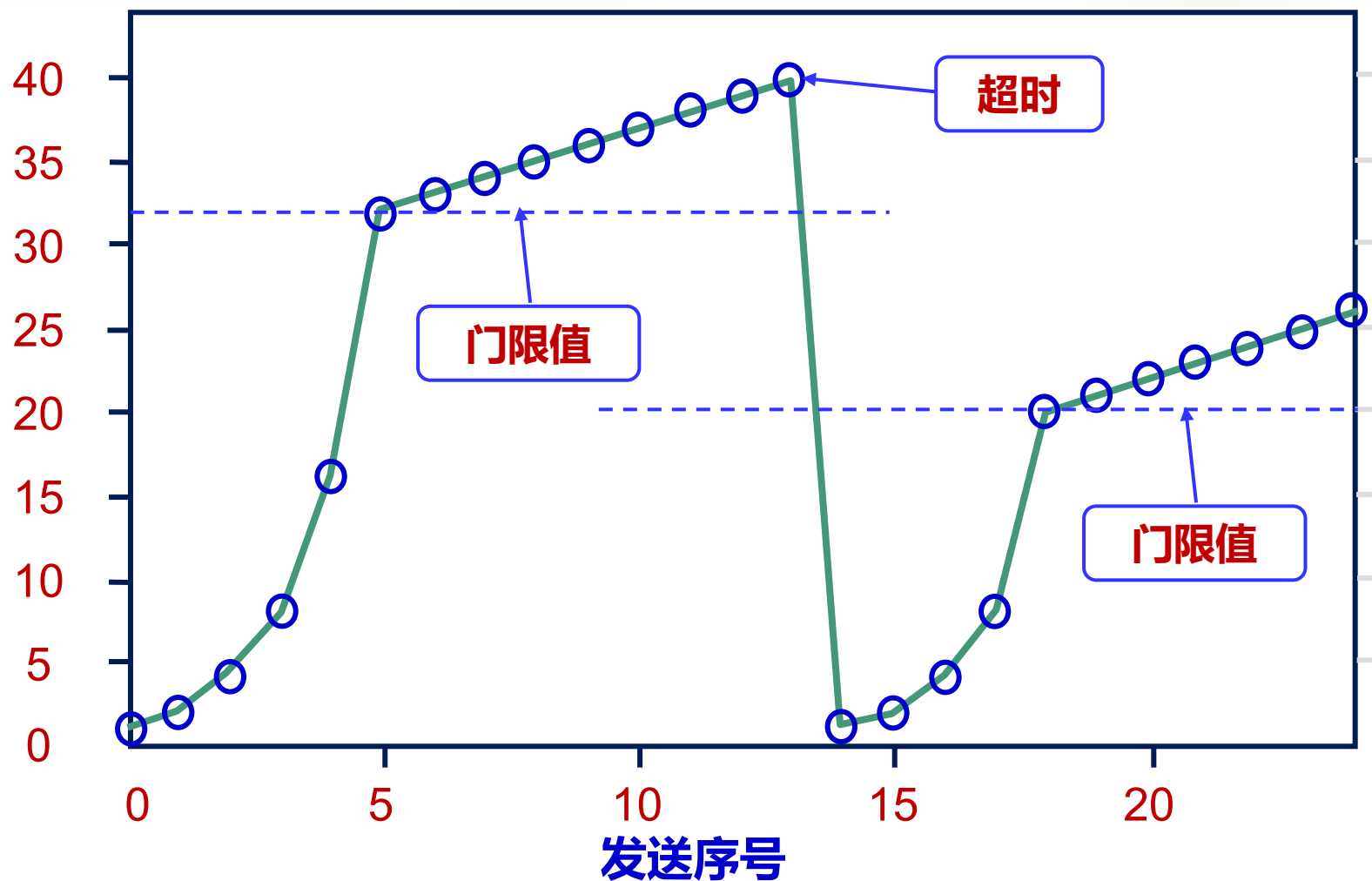
❖ 为避免窗口增加过快，TCP 还采用了拥塞避免算法

- ✓ 当拥塞窗口的大小增加上一次拥塞时窗口大小的一半时，TCP 进入拥塞避免状态，降低拥塞窗口增大的速度；
- ✓ 在拥塞避免状态中，只有在窗口中所有的报文段都被确认之后，拥塞窗口的大小才增加 1

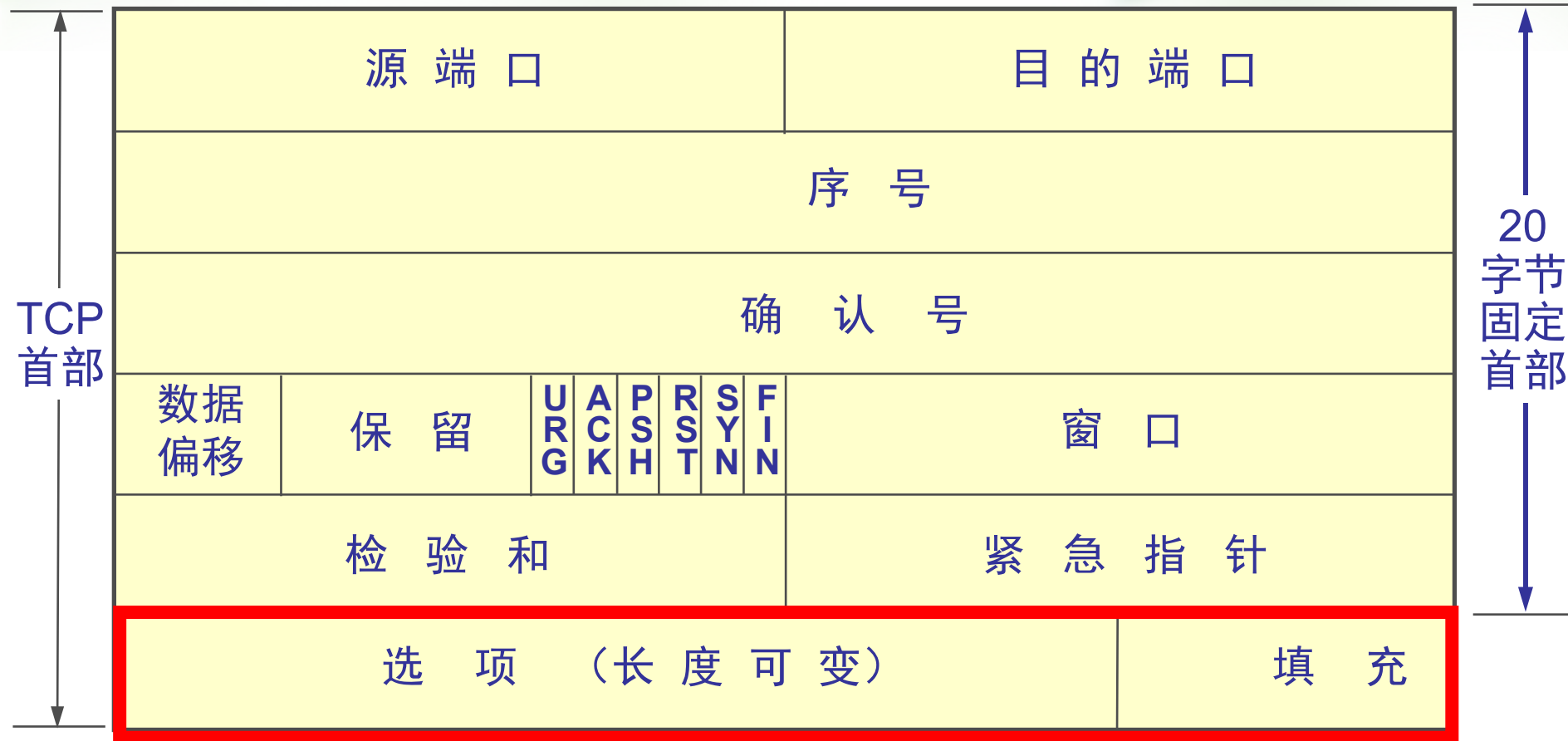


拥塞恢复

拥塞窗口大小 (KB)



位 0 8 16 24 31



选项字段

TCP 选项

❖ 用于连接建立阶段的协商

❖ MSS 选项（最大报文段长度）

✓ 配合通路MTU发现机制，来选择TCP的MSS

❖ 窗口扩大因子选项

✓ 支持发送窗口大于 64KB

❖ 时间戳选项

✓ 用于计算 RTT